

Day 2: Session 3

More Data Wrangling, Statistical Tests and Models

Session Objectives

1. Statistical Tests and Models
 - A. T-test and Bartlett's test of equal variance
 - B. Effectsizes - Cohen's d
 - C. Wilcox (Mann Whitney) Test
 - D. Chi-square test
 - E. ANOVA (Analysis of Variance)
 - F. Linear Regression
 - G. Logistic Regression
 2. Merging Datasets
 - A. joins (add columns)
 - B. concatenate (bind rows)
 3. Restructuring Datasets
 - A. Restructure from WIDE to LONG
 - B. Restructure from LONG to WIDE
-

0. Pework - Before You Begin

- R Packages Needed for this session:

- `dplyr`
 - `effectsize`

- * Note: `effectsize` is also part of a suite of packages called `easystats`.

- `gmodels`
 - `gtsummary`
 - `effects`
 - `emmeans`
 - `broom`
 - `tidyr`

- R Code Files

- `module_d2_01.R`

- SAS Code Files

- `sascode_d2_01.sas`

- R Data Files

- `help.Rdata`
 - `dat1.Rdata`
 - `dat2.Rdata`
 - `treat1.Rdata`
 - `treat2.Rdata`

- SAS Data Files

- `help.sas7bdat`
 - `dat1.sas7bdat`
 - `dat2.sas7bdat`
 - `treat1.sas7bdat`
 - `treat2.sas7bdat`

Notes from DAY 1 and Homework Review

- Saving Plots [saveplots.R](#)
- Homework 1 Answer Key [d1_homework_key.R](#)
- Data Import Preview Error - for earlier versions of RStudio (before June 2026)
 - Google Prompt “rstudio data preview could not find .rs.digest”
 - solution at [RStudio Github Issue](#)
 - rollback to earlier version of [RStudio Desktop](#)
 - see [RStudio Desktop Versions](#)
- look at `help(sd, package = "stats")` - scroll down to see also - `forvar()` function
- Note on new files - notice * at top or `untitled` - will prompt you to save on close
- Note/Tip - adding `====` 4 equals signs at the end of a comment `# xxxx` allows for “bookmarks” to be created in your code.
- Customizing decimal places

```
x <- 5.12345  
print(x, digits = 3)
```

```
[1] 5.12
```

- Summary stats for multiple variables - with `summary()`.

```
helpdata %>%  
  select(age, cesd1, racegrp) %>%  
  summary()
```

age		cesd1		racegrp	
Min.	:19.00	Min.	: 0.00	black	:211
1st Qu.	:30.00	1st Qu.	:11.00	hispanic	: 50
Median	:35.00	Median	:22.00	other	: 26
Mean	:35.65	Mean	:22.72	white	:166
3rd Qu.	:40.00	3rd Qu.	:34.75		
Max.	:60.00	Max.	:56.00		
		NAs	:207		

- Summary stats for multiple variables by group with `gtsummary`
 - learn more at [gtsummary::tbl_summary - vignette](#)

Characteristic	Control ¹	Treatment ¹	p-value ²
Age in years	36.30 (8.14)	35.00 (7.21)	0.071
CESD Depression Scores	32.86 (12.64)	32.84 (12.42)	0.990

¹Mean (SD)²Two Sample t-test

```
library(gtsummary)
helpdata %>%
  select(age, cesd, treat) %>%
  tbl_summary(
    by = treat,
    statistic = list(
      all_continuous() ~ "{mean} ({sd})"
    ),
    digits = all_continuous() ~ 2,
    label = list(
      age = "Age in years",
      cesd = "CESD Depression Scores"
    ) %>%
  modify_header(
    stat_1 = "**Control**",
    stat_2 = "**Treatment**"
  ) %>%
  bold_labels() %>%
  add_p(
    # perform t-test for all variables
    test = everything() ~ "t.test",
    # assume equal variance in the t-test
    test.args = all_tests("t.test") ~ list(var.equal = TRUE),
    pvalue_fun = label_style_pvalue(digits = 3)
  )
```

- Note on tibble output:

```
# get age stats by racegrp
racetbl <- helpdata %>%
  group_by(racegrp) %>%
  summarise(
    age_n = n(),
```

```
meanage = mean(age, na.rm=TRUE),
sdage = sd(age, na.rm=TRUE),
medage = median(age, na.rm=TRUE),
q25age = quantile(age,
                  probs = 0.25,
                  na.rm=TRUE),
q75age = quantile(age,
                  probs = 0.75,
                  na.rm=TRUE),
minage = min(age, na.rm=TRUE),
maxage = max(age, na.rm=TRUE)
)

# what is the output data/table?
class(racetbl)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

```
racetbl
```

```
# A tibble: 4 x 9
```

	racegrp	age_n	meanage	sdage	medage	q25age	q75age	minage	maxage
	<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<int>
1	black	211	35.7	7.08	35	31	39	20	60
2	hispanic	50	33.2	7.99	32	28.2	36.2	21	55
3	other	26	35.0	7.66	34	30	40.5	22	48
4	white	166	36.5	8.28	36	30	42	19	58

1. Statistical Tests and Models

Another operator to learn ~ (tilde)

So far you have learned about:

- `<-` the assignment operator for saving (“assigning”) output into an R object, (*e.g.* `x <- 5`) and
- `=` the equals sign to assign values to arguments or parameters to be defined within a function, (*e.g.* `seq(from=1, to=10, by=0.5)`).

Another operator used in R, is the “tilde” character `~` (which is just below my “ESC” key on my computer keyboard). The `~` tilde is often used for these two purposes:

1. in formulas - for example, to create a regression model, we will write:

$$y \sim x$$

which can read as “model Y as a function of X”, where “X” can be a single variable or a longer more complicated function.

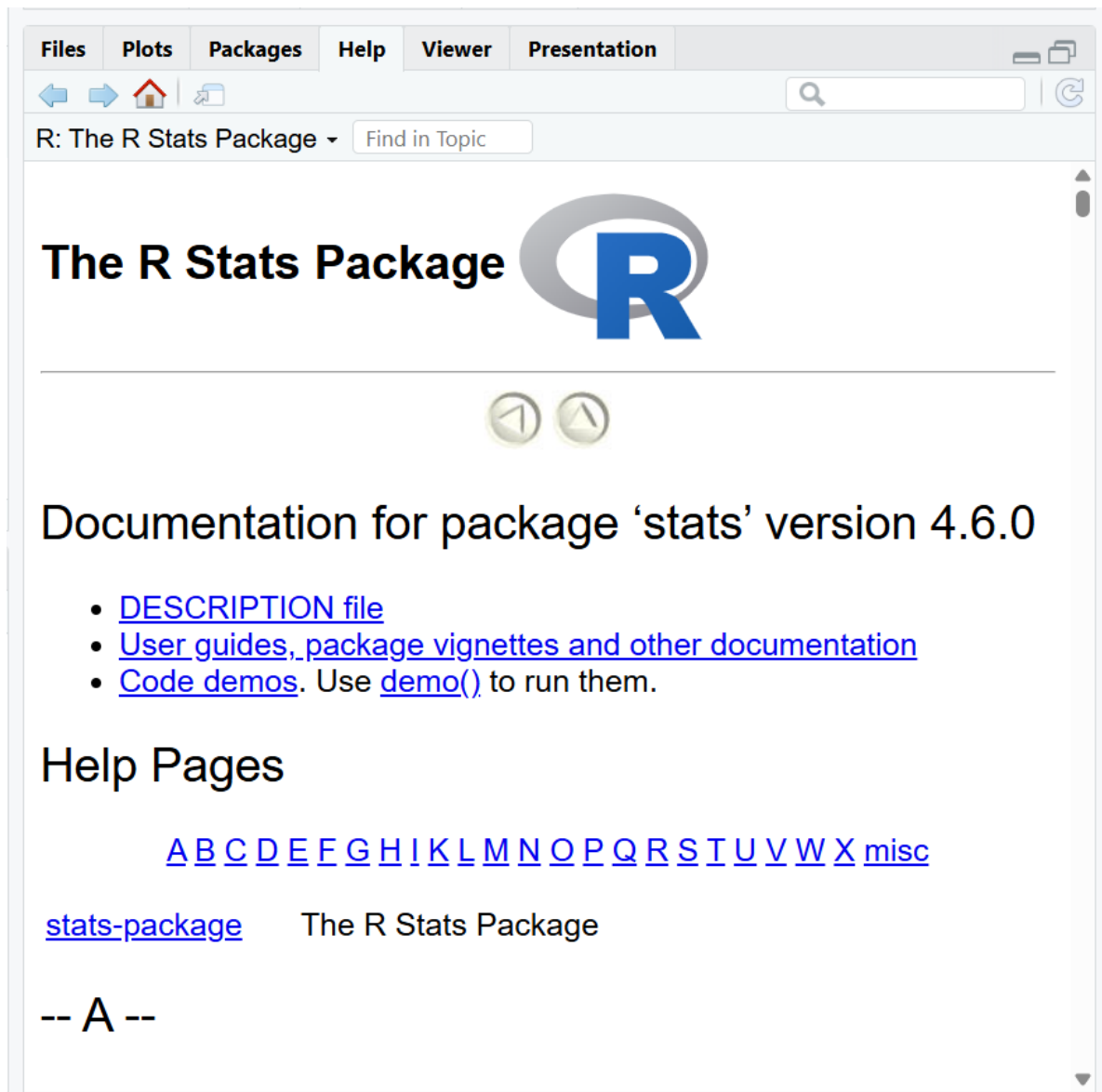
2. for assignment, which can still sort of be interpreted as “as a function of”. You will see this use of `~` in:
 - the `case_when()` function from the `dplyr` package
 - defining variables for facets in `ggplot2`
 - assigning parameters for custom output using `tbl_summary()` from the `gtsummary` package

List of Builtin Statistical Tests in R

The `stats` package which is part of the base R installation, includes a number of helpful built-in statistical tests, including:

- `ansari.test` - Ansari-Bradley Test
- `bartlett.test` - Bartlett Test of Homogeneity of Variances
- `binom.test` - Exact Binomial Test
- `Box.test` - Box-Pierce and Ljung-Box Tests
- `chisq.test` - Pearson's Chi-squared Test for Count Data
- `cor.test` - Test for Association/Correlation Between Paired Samples
- `fisher.test` - Fisher's Exact Test for Count Data
- `fligner.test` - Fligner-Killeen Test of Homogeneity of Variances
- `friedman.test` - Friedman Rank Sum Test
- `kruskal.test` - Kruskal-Wallis Rank Sum Test
- `ks.test` - Kolmogorov-Smirnov Tests
- `mantelhaen.test` - Cochran-Mantel-Haenszel Chi-Squared Test for Count Data
- `mauchly.test` - Mauchly's Test of Sphericity
- `mcnemar.test` - McNemar's Chi-squared Test for Count Data
- `mood.test` - Mood Two-Sample Test of Scale
- `oneway.test` - Test for Equal Means in a One-Way Layout
- `pairwise.prop.test` - Pairwise comparisons for proportions
- `pairwise.t.test` - Pairwise t tests
- `pairwise.wilcox.test` - Pairwise Wilcoxon Rank Sum Tests
- `poisson.test` - Exact Poisson tests
- `PP.test` - Phillips-Perron Test for Unit Roots
- `prop.test` - Test of Equal or Given Proportions
- `prop.trend.test` - Test for trend in proportions
- `quade.test` - Quade Test
- `shapiro.test` - Shapiro-Wilk Normality Test
- `t.test` - Student's t-Test
- `var.test` - F Test to Compare Two Variances
- `wilcox.test` - Wilcoxon Rank Sum and Signed Rank Tests

We will only use a few of these today. Learn more by reading about the `stats` package under the “PACKAGES” tab and “HELP”.



A. T-test and Bartlett's test of equal variance

To perform a test for differences between two-independent samples, we will use the `t.test()` function.

Let's perform a two-group independent T-test to see if there were any significant differences between the control and HELP intervention treatment groups at 6 months for the CESD scores.

We have to supply:

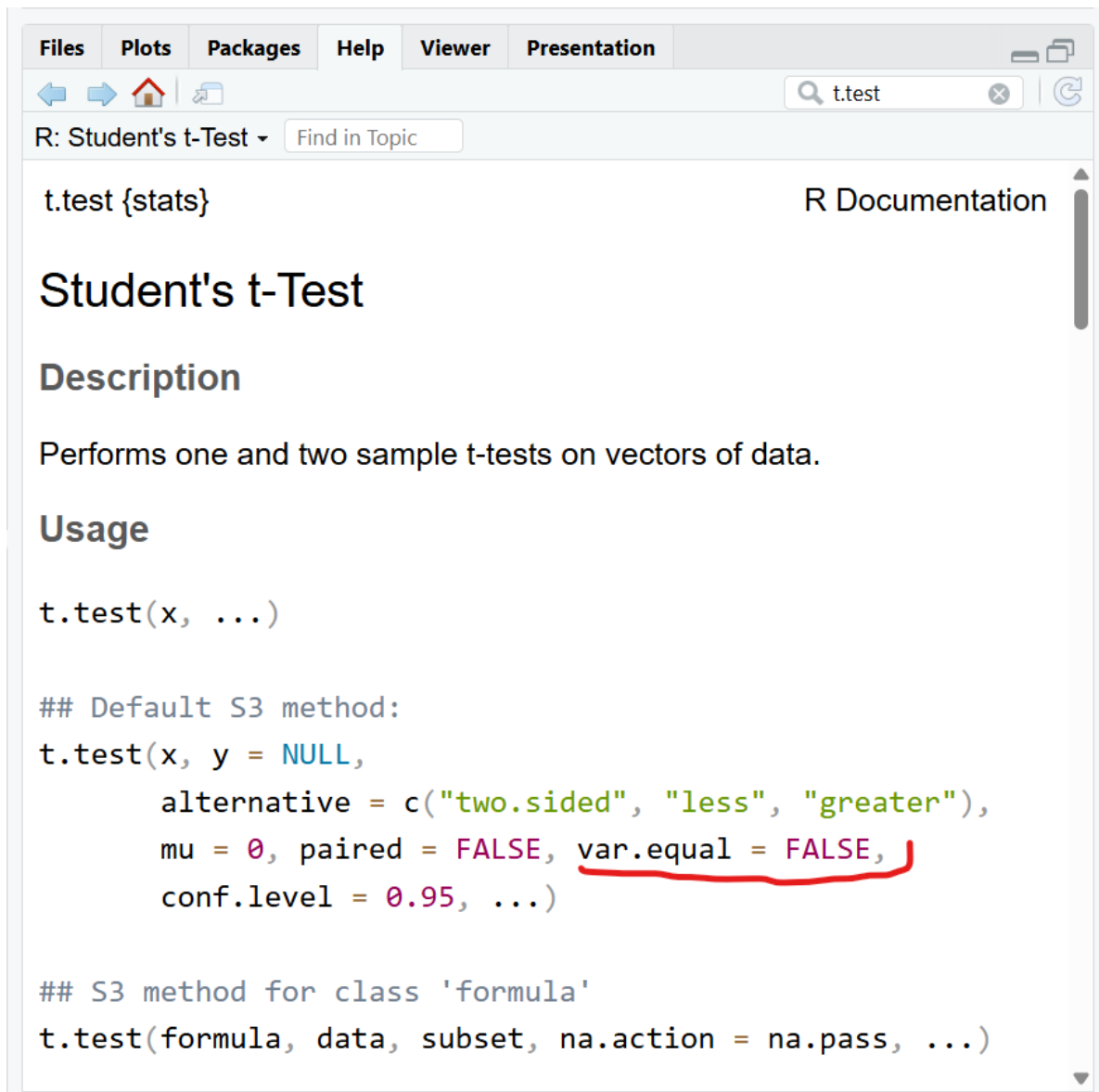
- the variable of interest `cesd1` for the CESD scores at 6 months
- the group variable `treat` which was coded 0=control and 1=HELP intervention treatment group
- and the dataset for these data `helpdata`

```
# compare cesd scores at 6m by treatment group
# NOTE: var.equal= FALSE is default
# unpooled t-test run by default
t.test(cesd1 ~ treat,
       data = helpdata)
```

Welch Two Sample t-test

```
data: cesd1 by treat
t = -0.48019, df = 243.97, p-value = 0.6315
alternative hypothesis: true difference in means between group 0 and group 1
is not equal to 0
95 percent confidence interval:
 -4.438530  2.698603
sample estimates:
mean in group 0 mean in group 1
    22.25217      23.12214
```

Notice that the default setting in `t.test()` for `var.equal` is `FALSE` which means that the UNPOOLED T-test is performed in R by default.



The screenshot shows the R Documentation window for the `t.test` function. The window has a menu bar with 'Files', 'Plots', 'Packages', 'Help', 'Viewer', and 'Presentation'. Below the menu bar is a search bar containing 't.test'. The main content area is titled 'R: Student's t-Test' and includes a 'Find in Topic' search box. The documentation text is as follows:

```
t.test {stats}

Student's t-Test

Description
Performs one and two sample t-tests on vectors of data.

Usage
t.test(x, ...)

## Default S3 method:
t.test(x, y = NULL,
       alternative = c("two.sided", "less", "greater"),
       mu = 0, paired = FALSE, var.equal = FALSE,
       conf.level = 0.95, ...)

## S3 method for class 'formula'
t.test(formula, data, subset, na.action = na.pass, ...)
```

The text 'R Documentation' is visible in the top right corner of the content area. The `var.equal = FALSE` line in the usage section is underlined in red.

To run a pooled T-test, we just need to set `var.equal=TRUE`.

```
# run pooled t-test
t.test(cesd1 ~ treat,
       var.equal = TRUE,
       data = helpdata)
```

Two Sample t-test

```
data: cesd1 by treat
t = -0.47578, df = 244, p-value = 0.6347
alternative hypothesis: true difference in means between group 0 and group 1
is not equal to 0
95 percent confidence interval:
 -4.471648  2.731721
sample estimates:
mean in group 0 mean in group 1
    22.25217      23.12214
```

While the default output from this function is useful, there is actually a lot of results generated by the `t.test()` function. Let's save all of this output and take a look.

```
# save results in an object
tt_cesd1 <-
  t.test(cesd1 ~ treat,
        data = helpdata)
```

The screenshot shows the RStudio interface with the following components:

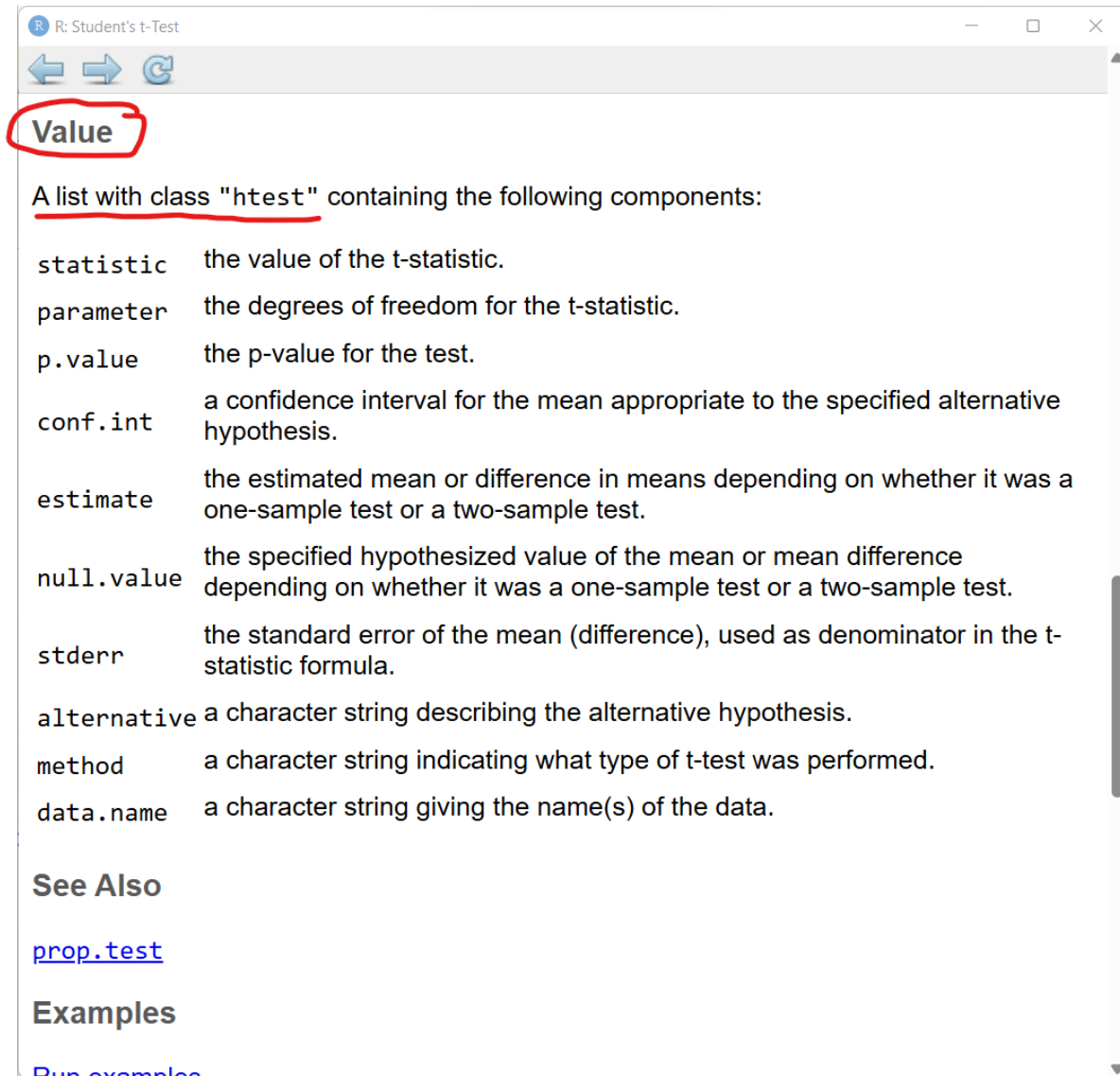
- Environment pane:** Lists objects in the Global Environment. `tt_cesd1` is highlighted with a red circle and a red arrow pointing to the console.
- Console pane:** Shows the execution of the following code:

```
> # save results in an object
> tt_cesd1 <-
+   t.test(cesd1 ~ treat,
+         data = helpdata)
> View(tt_cesd1)
> class(tt_cesd1)
[1] "htest"
```
- Environment table:** A table showing the attributes of the `tt_cesd1` object.

Name	Type	Value
tt_cesd1	list [10] (S3: htest)	List of length 10
statistic	double [1]	-0.4801922
parameter	double [1]	243.9673
p.value	double [1]	0.631521
conf.int	double [2]	-4.44 2.70
estimate	double [2]	22.3 23.1
null.value	double [1]	0
stderr	double [1]	1.811699
alternative	character [1]	'two.sided'
method	character [1]	'Welch Two Sample t-test'
data.name	character [1]	'cesd1 by treat'

The **R Documentation** pane on the right shows the documentation for `t.test`, including the description: "Performs one and two sample t-tests on vectors of data." and the usage: `t.test(x, ...)`.

We can also find out the expected output from the `t.test()` function by reading the “VALUE” section of the help page for `t.test`. Notice that `tt_cesd1` object we saved is of class “`htest`”.



The screenshot shows the R help page for the `t.test` function. The title bar reads "R: Student's t-Test". The "Value" section is circled in red. It describes the output as a list with class "htest" containing several components. Below this, a table lists the components and their descriptions. At the bottom, there are sections for "See Also" (with a link to `prop.test`) and "Examples" (with a link to "Run examples").

Value

A list with class "htest" containing the following components:

<code>statistic</code>	the value of the t-statistic.
<code>parameter</code>	the degrees of freedom for the t-statistic.
<code>p.value</code>	the p-value for the test.
<code>conf.int</code>	a confidence interval for the mean appropriate to the specified alternative hypothesis.
<code>estimate</code>	the estimated mean or difference in means depending on whether it was a one-sample test or a two-sample test.
<code>null.value</code>	the specified hypothesized value of the mean or mean difference depending on whether it was a one-sample test or a two-sample test.
<code>stderr</code>	the standard error of the mean (difference), used as denominator in the t-statistic formula.
<code>alternative</code>	a character string describing the alternative hypothesis.
<code>method</code>	a character string indicating what type of t-test was performed.
<code>data.name</code>	a character string giving the name(s) of the data.

See Also

[prop.test](#)

Examples

[Run examples](#)

Unfortunately, `t.test()` does not also automatically include a check for the assumption of equal variance to help decide whether you want the pooled or unpooled results. Instead you have to perform the test for equal variance separately.

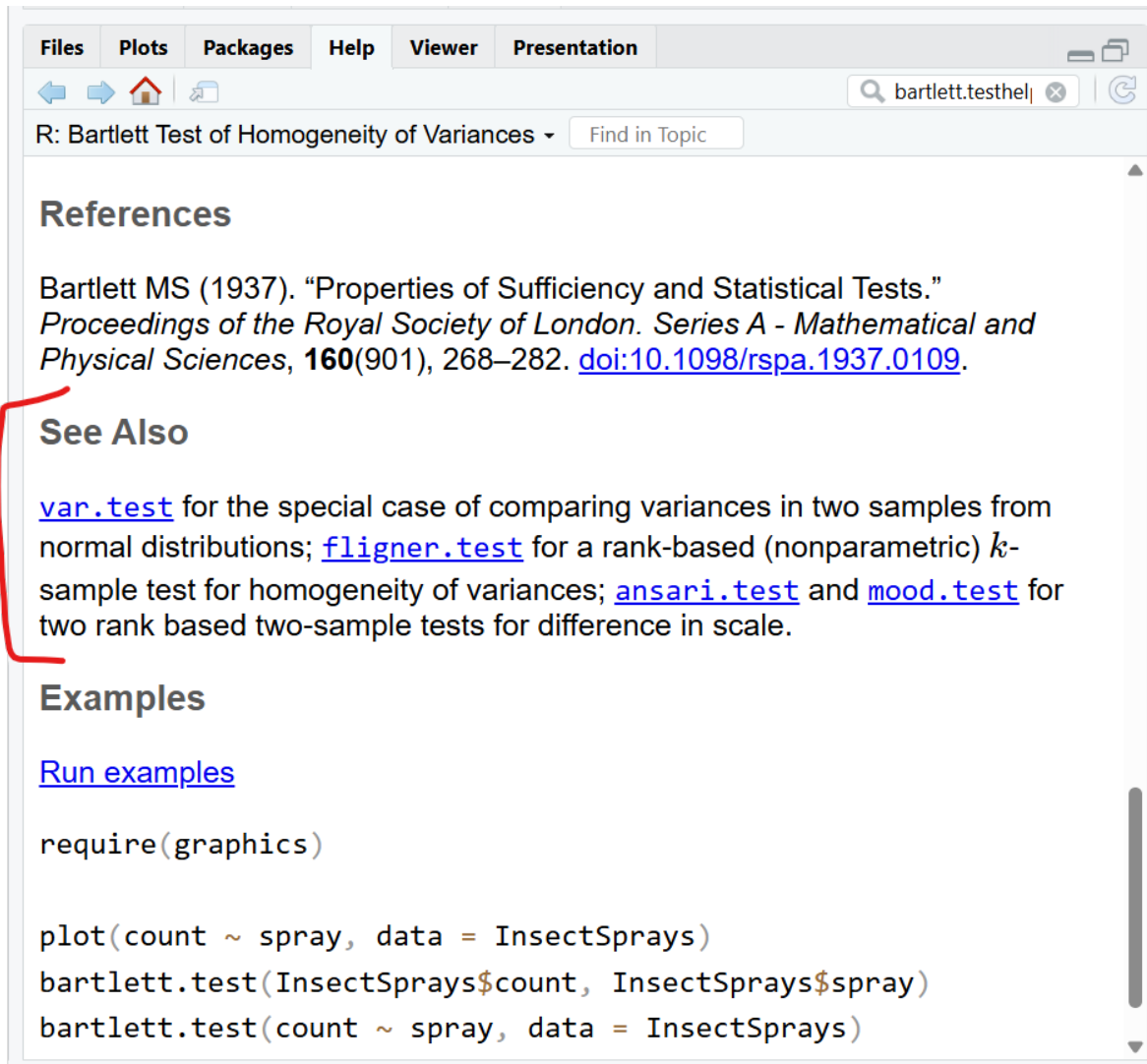
 Finding more options with related functions

It is useful to scroll through the entire help file for a given function. There is a section often towards the end labeled “See Also” that will usually link to other functions similar to or related to the function you are currently looking at.

I like running `bartlett.test()`. However, if we open the help page for `bartlett.test()`, we can scroll down to a section called “See Also” to see other functions that perform similar tests for equal variance including:

- `var.test` for the special case of comparing variances in two samples from normal distributions;
- `fligner.test` for a rank-based (nonparametric) k-sample test for homogeneity of variances;
- `ansari.test` and
- `mood.test` for two rank based two-sample tests for difference in scale.

```
help(bartlett.test, package = "stats")
```



The screenshot shows the R help window for the `bartlett.test` function. The window has a menu bar with 'Files', 'Plots', 'Packages', 'Help', 'Viewer', and 'Presentation'. Below the menu bar is a search bar containing 'bartlett.testhel' and a 'Find in Topic' button. The main content area is titled 'R: Bartlett Test of Homogeneity of Variances'. It contains a 'References' section with a citation for Bartlett MS (1937). Below this is a 'See Also' section, which is highlighted with a red bracket, containing links to `var.test`, `fligner.test`, `ansari.test`, and `mood.test`. The 'Examples' section follows, starting with a link to 'Run examples' and then showing three lines of R code: `require(graphics)`, `plot(count ~ spray, data = InsectSprays)`, `bartlett.test(InsectSprays$count, InsectSprays$spray)`, and `bartlett.test(count ~ spray, data = InsectSprays)`.

References

Bartlett MS (1937). "Properties of Sufficiency and Statistical Tests." *Proceedings of the Royal Society of London. Series A - Mathematical and Physical Sciences*, **160**(901), 268–282. [doi:10.1098/rspa.1937.0109](https://doi.org/10.1098/rspa.1937.0109).

See Also

[var.test](#) for the special case of comparing variances in two samples from normal distributions; [fligner.test](#) for a rank-based (nonparametric) k -sample test for homogeneity of variances; [ansari.test](#) and [mood.test](#) for two rank based two-sample tests for difference in scale.

Examples

[Run examples](#)

```
require(graphics)

plot(count ~ spray, data = InsectSprays)
bartlett.test(InsectSprays$count, InsectSprays$spray)
bartlett.test(count ~ spray, data = InsectSprays)
```

As you can see the test for variance differences is not statistically significant ($p > 0.05$), so we cannot reject the null hypothesis and conclude that the assumption of equal variance is met.

```
# check assumption of equal variance
bartlett.test(cesd1 ~ treat,
              data = helpdata)
```

Bartlett test of homogeneity of variances

```
data: cesd1 by treat
Bartlett's K-squared = 2.4304, df = 1, p-value = 0.119
```

Additionally, while the output for the `t.test()` above gives us the test statistic and p-value and the means for each group, it does not give us the standard deviations for each group, which is often helpful when communicating results.

So, we can use the `dplyr::summarise()` function to create these custom statistic and add `group_by()` to list the means and sd's for each group.

i package::function syntax

Once you load a package into your R computing session, you can just simply call the functions as you need them. However, occasionally packages might have the same function name which do very different things. For example, the `Hmisc` and `psych` packages both have a function called `describe()` for getting descriptive statistics which produce very different results. So, it can be helpful to call a function using its “formal” function call of `packagename::function()`, like `dplyr::summarise()` to avoid confusion.

```
# look at means and sd's of each group
# check to see if ratios of SDs is < 2
# which is another "rule of thumb" for
# checking the equal variance assumption
helpdata %>%
  group_by(treat) %>%
  summarise(
    meancesd1 = mean(cesd1, na.rm = TRUE),
    sdcesd1 = sd(cesd1, na.rm = TRUE)
  )
```

```
# A tibble: 2 x 3
  treat meancesd1 sdcesd1
  <int>     <dbl>   <dbl>
1     0      22.3    13.2
2     1      23.1    15.2
```

 Add sample sizes, counts of missing and non-missing

There are a couple of ways to get sample sizes with our `dplyr::summarise()` step. One is to use the `n()` function, but that will only show us the number of rows in the dataset for the given filter, regardless of the amount of missing values for a given variable. To get the amounts of missing (or non-missing), we can use the `is.na(varname)` function which returns a vector of `TRUE` and `FALSE` for `varname`. If we wrap `sum()` around that vector the `TRUE`'s are treated as 1's and the `FALSE`'s are 0's, so `sum(is.na(varname))` will return the number of `TRUE`'s or non-missing values. We can also use the `!` operator for the "logical NOT" operator, to get the amount of non-missing values using `sum(!is.na(varname))`.

Redo table above adding the sample size counts:

- the number of rows for each `treat` treatment group
- the amount of non-missing values for the CESD at 6m (`cesd1`)
- the amount of missing values for the CESD at 6m (`cesd1`)

```
# OPTIONAL - Sample Sizes
helpdata %>%
  group_by(treat) %>%
  summarise(
    all_n = n(),
    n_cesd1 = sum(!is.na(cesd1)),
    miss_cesd1 = sum(is.na(cesd1)),
    meancesd1 = mean(cesd1, na.rm = TRUE),
  )
```

```
sdcesd1 = sd(cesd1, na.rm = TRUE)
)
```

```
# A tibble: 2 x 6
  treat all_n n_cesd1 miss_cesd1 meancesd1 sdcesd1
  <int> <int>   <int>      <int>      <dbl>   <dbl>
1     0   228     115        113     22.3    13.2
2     1   225     131         94     23.1    15.2
```

SAS Code for T-test

```
* t-test;
proc ttest data=help;
  class treat;
  var cesd1;
run;
```

B. Effectsizes - Cohen's d

In addition to reporting the means and standard deviations of the two-groups, plus the test statistic, degrees of freedom and p-value, it is also helpful to report the effect sizes for tests and models as well. For a T-test, we could compute the effectsize Cohen's d to report along with our results.

There is a good package called `effectsize` that will allow us to compute Cohen's d using nearly identical syntax to the `t.test()`.

```
library(effectsize)
cohens_d(cesd1 ~ treat,
         pooled_sd = TRUE,
         data = helpdata)
```

Cohen's d	95% CI

-0.06	[-0.31, 0.19]

- Estimated using pooled SD.

i Another useful suite of packages, `easystats`

The `effectsize` package is part of another useful suite of packages called `easystats`.

C. Wilcox (Mann Whitney) Test

If we have non-normal data or other reasons for running a non-parametric version of a T-test, we can use the `wilcox.test()` function to perform a two-independent sample group test, which is also commonly known as the “Mann Whitney Test”, see `help(wilcox.test, package = "stats")`.

```
# run non-parametric two-group test =====  
wilcox.test(cesd1 ~ treat,  
            data = helpdata)
```

Wilcoxon rank sum test with continuity correction

data: cesd1 by treat

W = 7453.5, p-value = 0.8879

alternative hypothesis: true location shift is not equal to 0

SAS code for non-parametric two-group independent test

```
* non-parametric two-group test;  
proc npar1way data=help wilcoxon;  
  class treat;  
  var cesd1;  
run;
```

D. Chi-square test

We saw yesterday that we could get a Chi-Square Test performed using the `CrossTable()` function from the `gmodels` package and also using `tbl_cross()` from the `gtsummary` package. However, there is also a simple `chisq.test()` function in the base R `stats` package.

i Note on default settings

Be sure to read the help pages `help(chisq.test, package = "stats")` and note that the `chisq.test()` performs a continuity correction by default.

Since we've looking at differences above between the [CESD scores](#) at 6 months between the control and HELP intervention groups, let's use the recommended split of CESD scores at 16 and then use these two dichotomous categories to perform a Chi-square test of independence for `cesd1 > 16` versus `cesd1 <= 16`.

```
# compute cesd1 > 16;
helpdata <- helpdata %>%
  mutate(
    cesd1_gt16 = cesd1 > 16
  )
```

Use this new variable `cesd1_gt16` for the `chisq.test()`.

```
# built-in chisq.test() function
# Yates continuity correction TRUE by default
chisq.test(
  helpdata$cesd1_gt16,
  helpdata$treat
)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data:  helpdata$cesd1_gt16 and helpdata$treat
X-squared = 0.14403, df = 1, p-value = 0.7043
```

We could also perform this test using `gmodels::CrossTable()`.

```
library(gmodels)
gmodels::CrossTable(helpdata$cesd1_gt16,
  helpdata$treat,
  expected=TRUE,
  prop.r=FALSE,
  prop.t=FALSE,
  prop.chisq=FALSE,
  chisq=TRUE,
  fisher=TRUE,
  format = "SAS")
```

Cell Contents

	N
Expected N	
N / Col Total	

Total Observations in Table: 246

		helpdata\$treat		
helpdata\$cesd1_gt16		0	1	Row Total
FALSE		42	52	94
		43.943	50.057	
		0.365	0.397	
TRUE		73	79	152
		71.057	80.943	
		0.635	0.603	
Column Total		115	131	246
		0.467	0.533	

Statistics for All Table Factors

Pearson's Chi-squared test

Chi^2 = 0.2611261 d.f. = 1 p = 0.6093475

Pearson's Chi-squared test with Yates' continuity correction

Chi^2 = 0.1440294 d.f. = 1 p = 0.7043076

Fisher's Exact Test for Count Data

Sample estimate odds ratio: 0.8745577

Alternative hypothesis: true odds ratio is not equal to 1

p = 0.6933767

95% confidence interval: 0.5042603 1.512293

Alternative hypothesis: true odds ratio is less than 1

p = 0.3524224

95% confidence interval: 0 1.392588

Alternative hypothesis: true odds ratio is greater than 1

p = 0.7395405

95% confidence interval: 0.548163 Inf

We could also create a nice factor type variable with helpful labels for these crosstab tables as well. Let's make one and use it with `gtsummary::tbl_summary()` which can also perform a Chi-square or Fisher's exact test.

```
# add factor variable
helpdata$cesd1_gt16.f <-
  factor(
    helpdata$cesd1_gt16,
    levels = c(FALSE, TRUE),
    labels = c("CESD <= 16",
               "CESD > 16")
  )
```

```
library(gtsummary)
# option 1 with tbl_summary()
helpdata %>%
  select(cesd1_gt16.f, treat) %>%
  tbl_summary(
    by = treat,
    label = list(
      cesd1_gt16.f = "CESD Split at 16")
  ) %>%
  modify_header(
    stat_1 = "**Control**",
    stat_2 = "**Treatment**"
  ) %>%
  bold_labels() %>%
  add_p(
    test = cesd1_gt16.f ~ "chisq.test",
    test.args = list(cesd1_gt16.f ~ list(correct = TRUE)),
    pvalue_fun = label_style_pvalue(digits = 3)
  )
```

Table 1

Characteristic	Control ¹	Treatment ¹	p-value ²
CESD Split at 16			0.704
CESD ≤ 16	42 (37%)	52 (40%)	
CESD > 16	73 (63%)	79 (60%)	
Unknown	113	94	

¹n (%)²Pearson's Chi-squared test

! Handling missing values for Chi-square tests

It turns out that within `gtsummary` the two options for making these tables and running the tests of independence for these categorical variables, produce different results. By default, `chisq.test()`, `gmodels::CrossTable()` and `gtsummary::tbl_summary()` take out the missing data first and then run the Chi-square test. **BUT** `gtsummary::tbl_cross()` treats the missing values as their own “unknown category which changes the test results.

Compare missing data handling for `gtsummary::tbl_cross()`.

Notice the row for “unknown” for the missing values.

```
# option 2 with tbl_cross()
helpdata %>%
  tbl_cross(row = cesd1_gt16.f,
            col = treat,
            percent = "column",
            digits = c(0, 2),
            label = list(
              cesd1_gt16.f = "CESD Split at 16",
              treat = "Treatment Group"
            )) %>%
  bold_labels() %>%
  add_p(test = "chisq.test",
        test.args = list(correct = TRUE),
        pvalue_fun = label_style_pvalue(digits = 3))
```

Table 2

	Treatment Group		Total	p-value ¹
	0	1		
CESD Split at 16				0.220
CESD ≤ 16	42 (18.42%)	52 (23.11%)	94 (20.75%)	
CESD > 16	73 (32.02%)	79 (35.11%)	152 (33.55%)	
Unknown	113 (49.56%)	94 (41.78%)	207 (45.70%)	
Total	228 (100.00%)	225 (100.00%)	453 (100.00%)	

¹Pearson’s Chi-squared test

Remove the missing values with `filter(complete.cases(.))` and now the p-values match the results above.

```
# get all non-missing cases and re-run
h1 <- helpdata %>%
  select(cesd1_gt16.f, treat) %>%
  filter(complete.cases(.))

h1 %>%
  tbl_cross(row = cesd1_gt16.f,
            col = treat,
            percent = "column",
            digits = c(0, 2),
            label = list(
              cesd1_gt16.f = "CESD Split at 16",
              treat = "Treatment Group"
            )) %>%
  bold_labels() %>%
  add_p(test = "chisq.test",
        test.args = list(correct = TRUE),
        pvalue_fun = label_style_pvalue(digits = 3))
```

Table 3

	Treatment Group		Total	p-value ¹
	0	1		
CESD Split at 16				0.704
CESD ≤ 16	42 (36.52%)	52 (39.69%)	94 (38.21%)	
CESD > 16	73 (63.48%)	79 (60.31%)	152 (61.79%)	
Total	115 (100.00%)	131 (100.00%)	246 (100.00%)	

¹Pearson's Chi-squared test

SAS Code for Crosstables with Chi-square and Fisher's Exact tests

```
* compute cesd1 > 16;
data help;
  set help;
  if not missing(cesd1) then cesd1_gt16 = cesd1>16;
  else cesd1_gt16 = .;
run;

* chi-square test;
proc freq data=help;
  table cesd1_gt16 * treat / chisq fisher norow nopercent;
run;
```

E. ANOVA (Analysis of Variance)

In addition to comparisons of two-groups, let's also perform some comparisons of the 4 race/ethnicity groups with the `racegrp` variable to compare `mcs` (mental component scale from SF36 quality of life scale) scores at baseline.

We will use a similar formula syntax like above with the `~` tilde operator using the `aov()` function from the `stats` package.

$$y \sim x$$

We will perform the test and save the results in an output object called `fit.aov`. And then we will use this model output object as input to other functions to help us parse the model results.

```
# the aov() function
# gives the global test for the "group" effect
fit.aov <- aov(mcs ~ racegrp,
               data=helpdata)
fit.aov
```

Call:

```
aov(formula = mcs ~ racegrp, data = helpdata)
```

Terms:

	racegrp	Residuals
Sum of Squares	2732.82	71778.74
Deg. of Freedom	3	449

Residual standard error: 12.64372

Estimated effects may be unbalanced

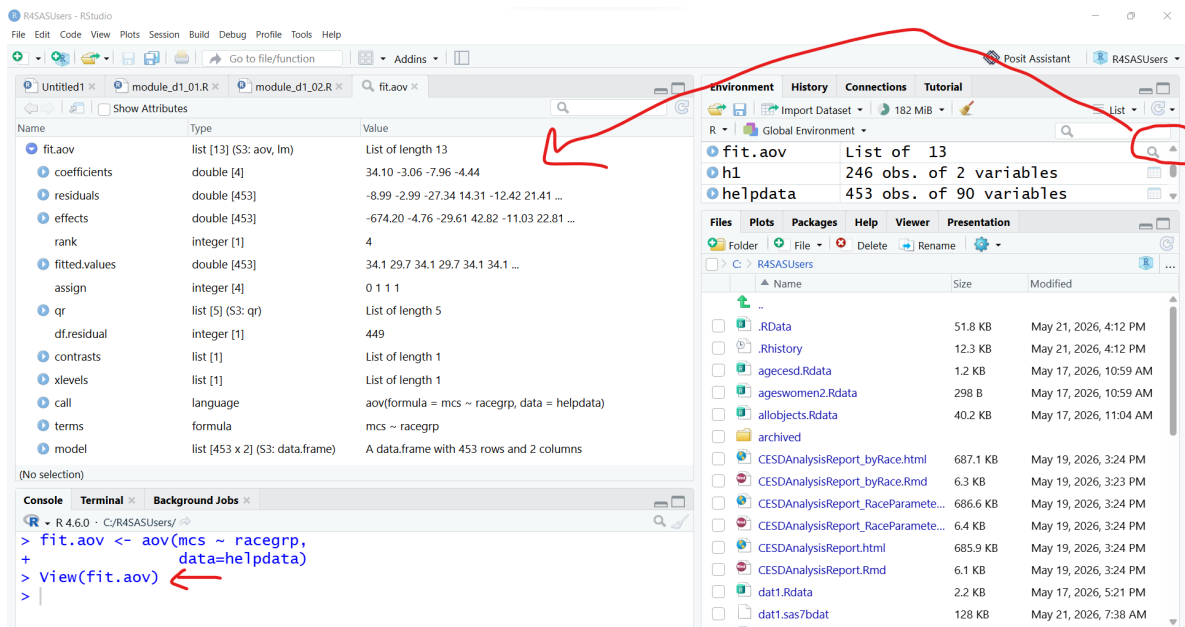
The default output is pretty basic. We only get the components of the F-test for the `racegrp` between subjects effect. But notice there is no p-value. How do we get that?

We need to use the `summary()` function again. Remember that I mentioned this is a “generic function” in R that “adapts” based on the kind of R object we input to `summary()`.

```
# what kind of object is fit.aov
class(fit.aov)
```

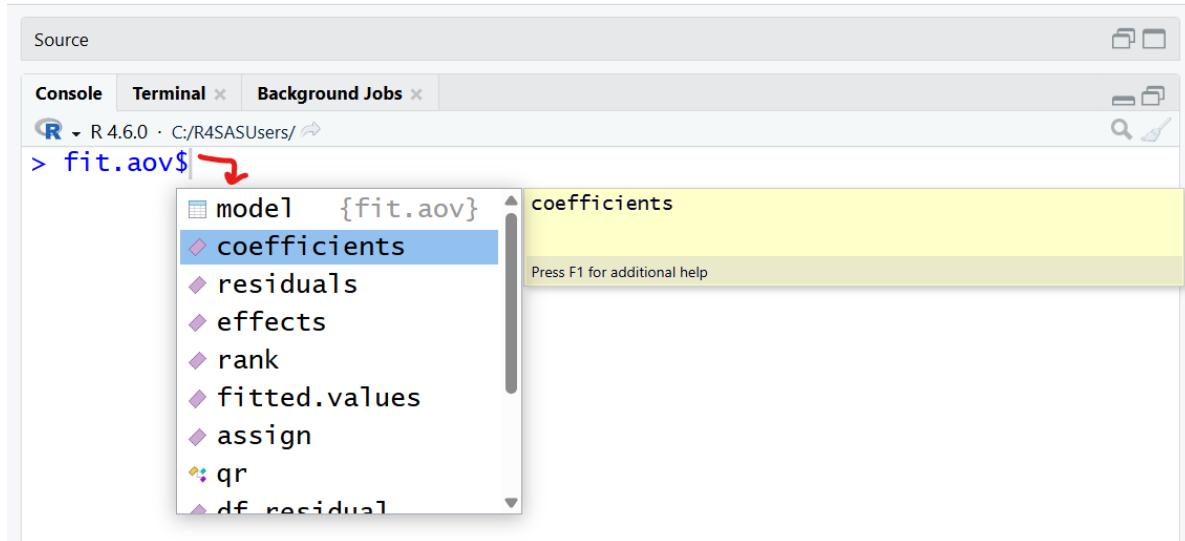
```
[1] "aov" "lm"
```

This model object is of class “aov” (analysis of variance) and “lm” (linear model). Let’s click on the magnifying glass icon in the “Global Environment” for the `fit.aov` object to see what is included with this object. You can also run the function `View(fit.aov)` which will also open the object viewer.



We can “look” at these elements inside the object in the viewer. Some of the elements are data.frames that contain other data element. For example, click on the table icon next to the line that says “coefficients”, this creates the code `fit.aov[["coefficients"]]` and when you hit return in the “Console” then displays the “linear model” coefficients from the ANOVA model.

Another way to select and display the elements within the `fit.aov` model is basically the same as selecting variables out of a `data.frame` dataset object.



Notice that these commands yield the same results:

```
fit.aov[["coefficients"]]
```

(Intercept)	racegrphspanic	racegrpoother	racegrpwhite
34.098940	-3.057568	-7.959072	-4.442592

```
fit.aov$coefficients
```

(Intercept)	racegrphspanic	racegrpoother	racegrpwhite
34.098940	-3.057568	-7.959072	-4.442592

The other thing to notice is that besides the (Intercept) term, the remaining 3 coefficients are:

- racegrphspanic
- racegrpoother
- racegrpwhite

Remember, the 4 race/ethnicity groups were: “black”, “hispanic”, “other” and “white” in alphabetical order. So, by default R chooses the FIRST category to be the reference category. This is why there are 3 coefficients - one for each pairwise comparison of the other 3 race/ethnicity groups to the “reference” category “black”. This is basically the same approach as creating dummy variable coding to essentially perform an ANOVA via linear regression.

There are 13 total elements inside of `fit.aov` include residuals and fitted values, for example.

Now that we understand a bit better what all is included within the `fit.aov` object, how can we get the F-test model results and p-values and such. We need to run `summary()` for the `fit.aov` object which technically runs `summary.aov()`, see help:

Files **Plots** **Packages** **Help** **Viewer** **Presentation**

summary.

summary.lm
summary.aov
summary.glm
summary.mlm
summary.nls
summary.stl
summary.ecdf
summary.Date
summary.table
summary.manova
summary.prcomp
summary.factor
summary.matrix
summary.srcref
summary.aovlist
summary.stepfun
summary.default
summary.POSIXct
summary.POSIXlt
summary.srcfile

R: Summarize an Analysis of Variance Model

summary.aov {stats}

Summarize an Analysis of Variance Model

Description

Summarize an analysis of variance model.

Usage

```
## S3 method for class 'aov'  
summary(object, intercept = FALSE, split = FALSE, expand.split = TRUE, keep.zeros = FALSE, ...)
```

Arguments

object An object of class "aov" or "aovlist".

The output “Value”’s from `summary.aov()` include sums of squares, degrees of freedom as well as the F-value and p-values “Pr(>F)”.

```
# get better formatted output
summary(fit.aov)
```

```
              Df Sum Sq Mean Sq F value    Pr(>F)
racegrp         3   2733    910.9    5.698 0.000778 ***
Residuals      449  71779    159.9
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

This finally gives us a helpful ANOVA output table with the F-test details which shows that for the MCS scores at baseline there are statistically significant differences between the 4 race/ethnicity groups: $F(3, 449) = 5.698$, $p = .000778$.

We could also have used the `anova()` generic function as well to get this ANOVA table:

```
anova(fit.aov)
```

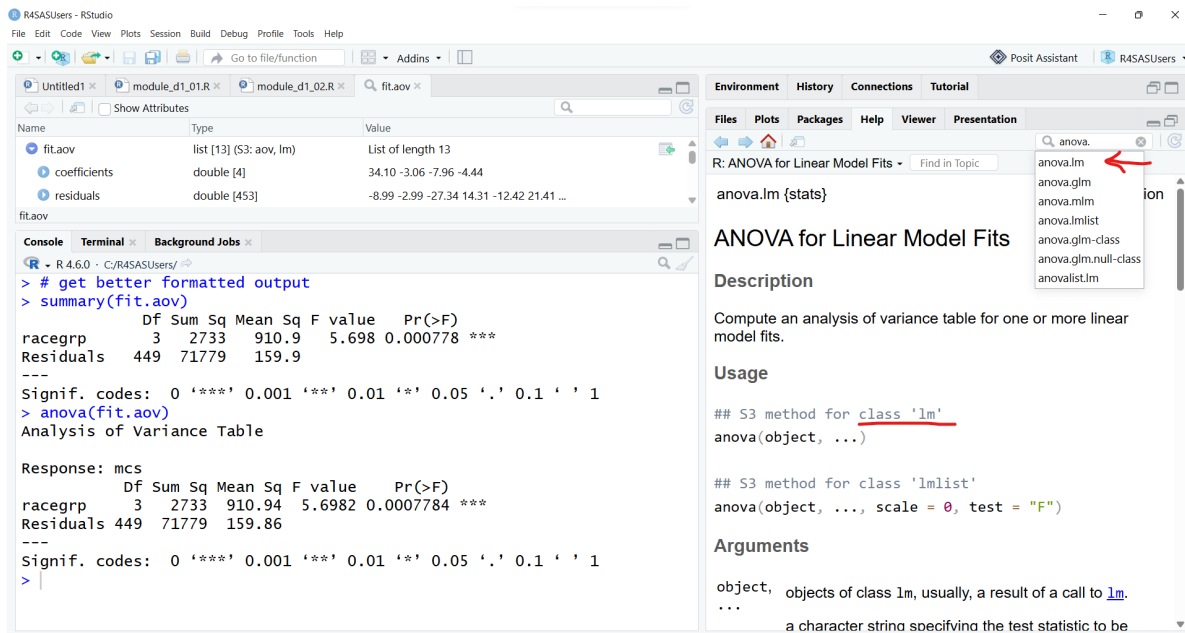
Analysis of Variance Table

Response: mcs

```
              Df Sum Sq Mean Sq F value    Pr(>F)
racegrp         3   2733    910.94    5.6982 0.0007784 ***
Residuals      449  71779    159.86
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Help window for `anova.lm()`:


The screenshot shows the RStudio interface. The console window displays the following output:

```
> # get better formatted output
> summary(fit.aov)
              Df Sum Sq Mean Sq F value    Pr(>F)    
racegrp        3    2733    910.94   5.698 0.000778 ***
Residuals    449   71779    159.9                
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

> anova(fit.aov)
Analysis of Variance Table

Response: mcs
              Df Sum Sq Mean Sq F value    Pr(>F)    
racegrp        3    2733    910.94   5.6982 0.0007784 ***
Residuals    449   71779    159.86                
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>
```

The help window for `anova.lm()` is open, showing the following information:

- Description:** Compute an analysis of variance table for one or more linear model fits.
- Usage:**

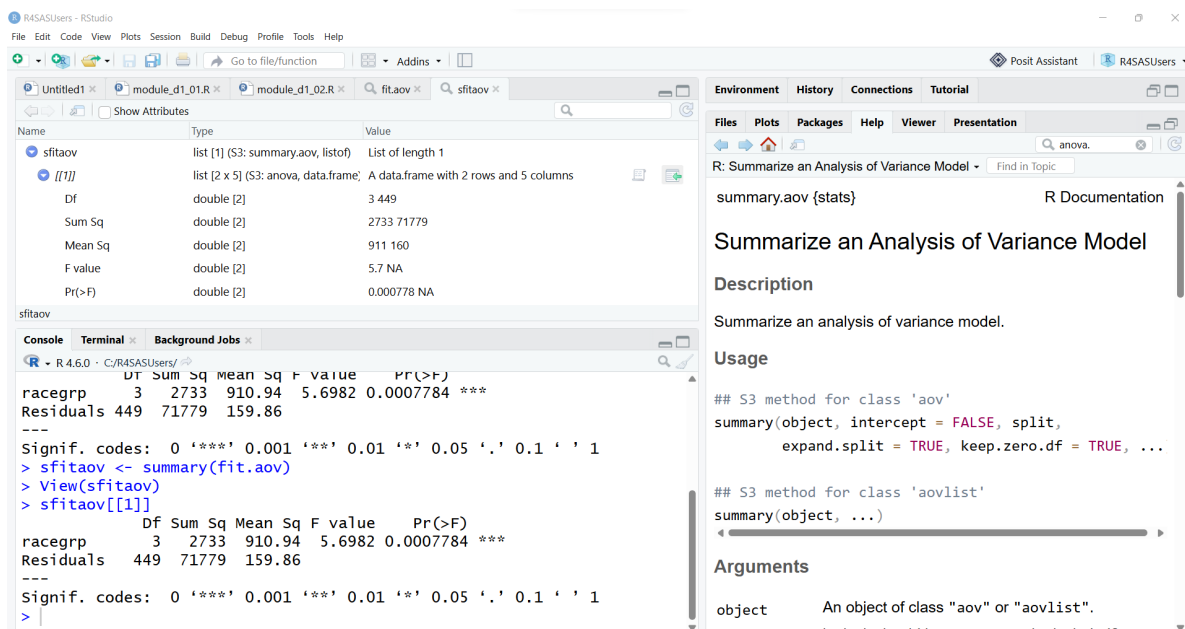
```
## S3 method for class 'lm'
anova(object, ...)
```

```
## S3 method for class 'lmlist'
anova(object, ..., scale = 0, test = "F")
```
- Arguments:**
 - `object`, objects of class `lm`, usually, a result of a call to `lm`.
 - `...`, a character string specifying the test statistic to be

Let's save the `summary.aov()` output object and take a look at it as well:

```
sfitaoav <- summary(fit.aov)
```

This object is basically a `data.frame` that has the elements of the ANOVA table.



The screenshot shows the RStudio interface. The console window displays the following output:

```
> summary(fit.aov)
              Df Sum Sq Mean Sq F value    Pr(>F)    
racegrp        3    2733    910.94   5.6982 0.0007784 ***
Residuals    449   71779    159.86                
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

> View(sfitaoav)
> sfitaoav[[1]]
              Df Sum Sq Mean Sq F value    Pr(>F)    
racegrp        3    2733    910.94   5.6982 0.0007784 ***
Residuals    449   71779    159.86                
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>
```

The help window for `summary.aov()` is open, showing the following information:

- Description:** Summarize an analysis of variance model.
- Usage:**

```
## S3 method for class 'aov'
summary(object, intercept = FALSE, split,
        expand.split = TRUE, keep.zero.df = TRUE, ...)
```

```
## S3 method for class 'aovlist'
summary(object, ...)
```
- Arguments:**
 - `object`, An object of class `"aov"` or `"aovlist"`.

For example, the p-value can be pulled out by running either of these commands:

The screenshot shows the RStudio interface with the `sfitaov` object selected in the Environment pane. The object is a list with a single element `[[1]]`, which is a data frame containing ANOVA results. A red circle highlights the copy icon in the Environment pane, and a red arrow points to the R console where the command `> sfitaov[[1]][["Pr(>F)"]]` is entered. Below the console, a tooltip for the `Pr(>F)` element is shown, displaying the value `0.0007784242` and the `NA` value. The console output shows the result of the command: `[1] 0.0007784242` and `NA`.

Name	Type	Value
sfitaov	list [1] (S3: summary.aov, listof)	List of length 1
[[1]]	list [2 x 5] (S3: anova, data.frame)	A data.frame with 2 rows and 5 columns
Df	double [2]	3 449
Sum Sq	double [2]	2733 71779
Mean Sq	double [2]	911 160
F value	double [2]	5.7 NA
Pr(>F)	double [2]	0.000778 NA

```

> sfitaov[[1]][["Pr(>F)"]]
[1] 0.0007784242
[1] NA

```

```
sfitaov[[1]][["Pr(>F)"]]
```

```
[1] 0.0007784242      NA
```

```
sfitaov[[1]]$`Pr(>F)`
```

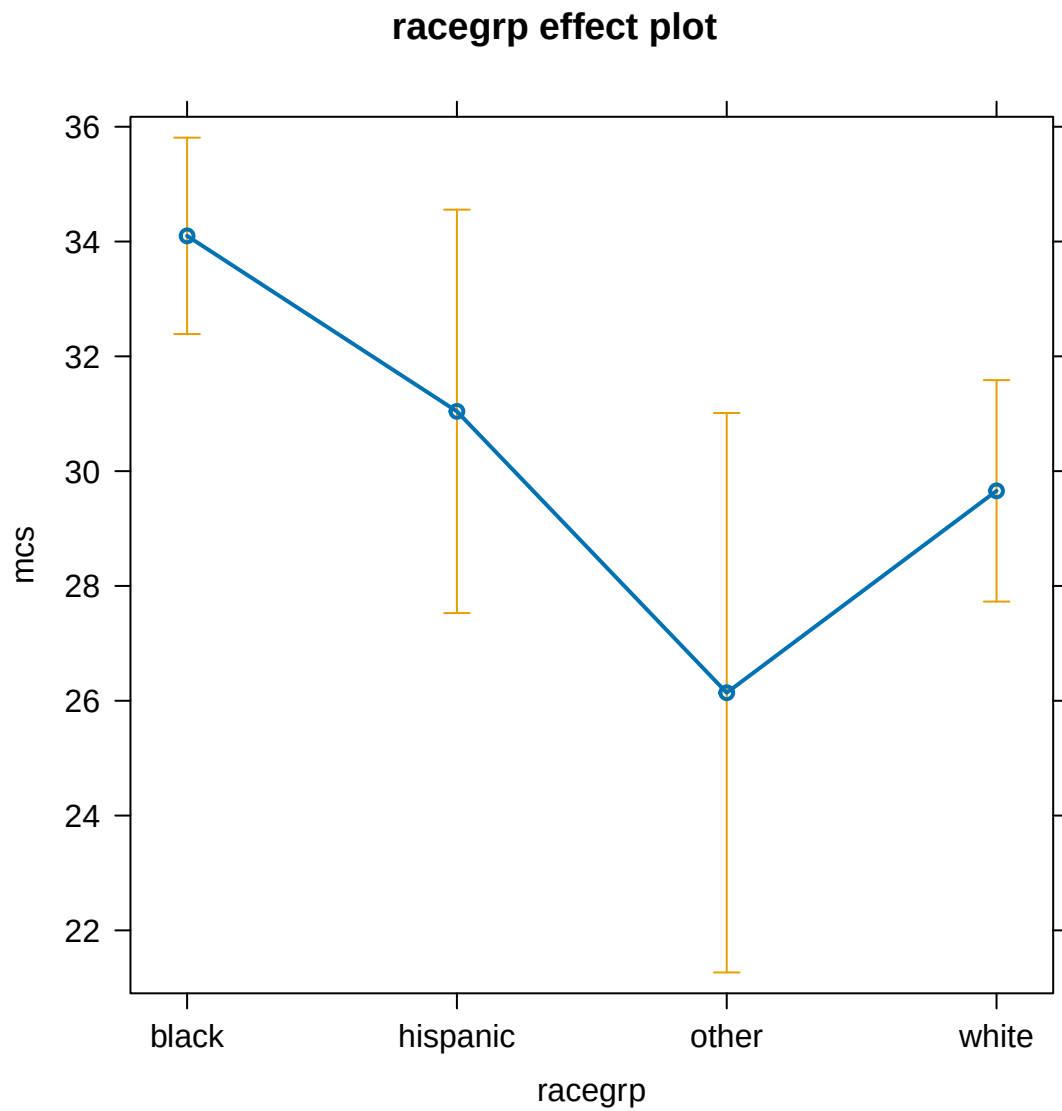
```
[1] 0.0007784242      NA
```

Let's also take a look at some ways to visualize the “effects” from the ANOVA model. One package that is helpful to visualize model results is the **effects** package. Let's use the `allEffects()` function on this model output `fit.aov` and make a plot of these effects:

```
# use effects package to get
# effects plots - basically a plot
# of the means with 95% confidence intervals
# for each group, racegrp

library(effects)

# make the effects plot
plot(allEffects(fit.aov))
```



We should also run the pairwise posthoc tests to see where the differences lie between these 4 race/ethnicity groups. There are a few ways to do this, but one package that is helpful is the `emmeans` package. For now, I'll use a the Bonferroni alpha-inflation correction for multiple pairwise tests. You can see the other correction options by viewing `help(summary.emmGrid, package="emmeans")` - scroll down to the "P-Value Adjustments" section to learn more.

```
# get post hoc tests
# to see the list of p-value adjustments for
# multiple comparisons type
# help(summary.emmGrid, package="emmeans")
```

```
library(emmeans)
# bonferroni p-value adjustments
emmeans(fit.aov,
         specs = pairwise ~ racegrp,
         adjust = "bonferroni")
```

```
$emmeans
  racegrp  emmean    SE  df lower.CL upper.CL
black      34.1 0.870 449    32.4    35.8
hispanic   31.0 1.790 449    27.5    34.6
other      26.1 2.480 449    21.3    31.0
white      29.7 0.981 449    27.7    31.6
```

Confidence level used: 0.95

```
$contrasts
  contrast      estimate    SE  df t.ratio p.value
black - hispanic     3.06 1.99 449   1.537 0.7493
black - other        7.96 2.63 449   3.029 0.0156
black - white        4.44 1.31 449   3.387 0.0046
hispanic - other     4.90 3.06 449   1.603 0.6574
hispanic - white     1.39 2.04 449   0.679 1.0000
other - white       -3.52 2.67 449  -1.319 1.0000
```

P value adjustment: bonferroni method for 6 tests

Given these Bonferroni adjusted p-values, the statistically significant ($p < .05$) differences are between "black - other" ($p=.0156$) and between "black - white" ($p=.0046$).

And finally it would be good to still check the assumption of equality of variance for the ANOVA. I'll use `bartlett.test()` again. The result is a non-significant p-value, so I'll conclude the assumption of equality of variance is met.

```
# bartlett's test for homogeneity of variances
# note: put the formula back in
bartlett.test(mcs ~ racegrp,
              data=helpdata)
```

Bartlett test of homogeneity of variances

```
data: mcs by racegrp
Bartlett's K-squared = 5.2581, df = 3, p-value = 0.1538
```

Unequal group variances - Welch's test

If you do end up with a statistically significant equality of variance test and need to run an adjusted ANOVA, there is the `oneway.test()` function from the `stats` package.

```
# One-way ANOVA assuming equal variance
oneway.test(mcs ~ racegrp, var.equal=TRUE, data=helpdata)

# One-way ANOVA assuming unequal variance = Welch's test
oneway.test(mcs ~ racegrp, var.equal=FALSE, data=helpdata)

# Learn more
help(oneway.test, package = "stats")
```

SAS Code for ANOVA

```
* ANOVA
compare mcs between 4 race groups
run bonferroni adjusted pairwise comparisons
run homogeneity of variance
and get welch's adjusted test if needed;

proc ANOVA data=help;
  class racegrp;
  model mcs = racegrp;
  means racegrp / bon hovtest welch;
run;
```

F. Linear Regression

Linear regression approach to ANOVA

As we saw above, the analysis of variance model object is of both class `aov` and `lm`. You can actually use the `lm()` function to also perform an ANOVA for a group variable. You will get the same `anova()` table results.

```
# use lm to perform an ANOVA =====
# ANOVA
# the aov() function
# gives the global test for the "group" effect
fit.lmaov <- lm(mcs ~ racegrp,
                data=helpdata)
summary(fit.lmaov)
```

Call:

```
lm(formula = mcs ~ racegrp, data = helpdata)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-27.336	-9.408	-2.049	9.399	32.519

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	34.0989	0.8704	39.175	< 2e-16 ***
racegrp hispanic	-3.0576	1.9887	-1.537	0.124882
racegrp other	-7.9591	2.6280	-3.029	0.002599 **
racegrp white	-4.4426	1.3117	-3.387	0.000769 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 12.64 on 449 degrees of freedom

Multiple R-squared: 0.03668, Adjusted R-squared: 0.03024

F-statistic: 5.698 on 3 and 449 DF, p-value: 0.0007784

```
anova(fit.lmaov)
```

Analysis of Variance Table

Response: mcs

```
      Df Sum Sq Mean Sq F value    Pr(>F)
racegrp   3   2733   910.94   5.6982 0.0007784 ***
Residuals 449  71779   159.86
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
sfitlmaov <- summary(fit.lmaov)
```

Let's make some nicer tables for these results using the `tbl_regression()` function from the `gtsummary` package.

The following code chunks show how to build the regression table elements.

The basic table:

```
# formatted output using tbl_summary()
# from gtsummary package
# racegrp sorted alphabetically
# first group "black" used a reference
fit.lmaov %>%
  tbl_regression()
```

Table 4

Characteristic	Beta	95% CI	p-value
racegrp			
black	—	—	
hispanic	-3.1	-7.0, 0.85	0.12
other	-8.0	-13, -2.8	0.003
white	-4.4	-7.0, -1.9	<0.001

Abbreviation: CI = Confidence Interval

Add the overall p-value results for the whole ANOVA table (e.g. the **racegrp** effect).

```
# add global p-value  
# for racegrp as a whole  
fit.lmaov %>%  
  tbl_regression() %>%  
  add_global_p()
```

Table 5

Characteristic	Beta	95% CI	p-value
racegrp			<0.001
black	—	—	
hispanic	-3.1	-7.0, 0.85	
other	-8.0	-13, -2.8	
white	-4.4	-7.0, -1.9	

Abbreviation: CI = Confidence Interval

Finally add custom model fit statistics using the `add_glance_table()` function to add select model fit statistics to your regression results table. This function works in concert with the [broom package](#) which is part of the [tidymodels](#) suite of packages, which is explained below.

```
# add model statistics
fit.lmaov %>%
  tbl_regression() %>%
  add_glance_table(
    include = c(r.squared,
                adj.r.squared,
                p.value)
  )
```

Table 6

Characteristic	Beta	95% CI	p-value
racegrp			
black	—	—	
hispanic	-3.1	-7.0, 0.85	0.12
other	-8.0	-13, -2.8	0.003
white	-4.4	-7.0, -1.9	<0.001
R ²	0.037		
Adjusted R ²	0.030		
p-value	<0.001		

Abbreviation: CI = Confidence Interval

To see what model statistics are available, we need to look at what the `broom` package is able to pull out from the `fit.lmaov` model object.

The `broom::glance()` function computes many model fit statistics we can choose from to display as an add on option to `gtsummary::tbl_regression()` using the `add_glance_table()` function.

```
# model statistics available
library(broom)
bg <- broom::glance(fit.lmaov)
bg
```

```
# A tibble: 1 x 12
  r.squared adj.r.squared sigma statistic p.value    df logLik   AIC   BIC
    <dbl>         <dbl> <dbl>     <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>
1  0.0367         0.0302  12.6      5.70 0.000778     3 -1790. 3590. 3611.
# i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```
names(bg)
```

```
[1] "r.squared"      "adj.r.squared" "sigma"          "statistic"
[5] "p.value"        "df"             "logLik"         "AIC"
[9] "BIC"           "deviance"       "df.residual"    "nobs"
```

More traditional linear regression model

Let's stay with looking at the mcs at baseline and perform a regression model of the association between the mcs and cesd plus age and gender female.

```
# fit a regression model =====  
fit.lmreg <- lm(mcs ~ age + female + cesd,  
               data = helpdata)  
summary(fit.lmreg)
```

Call:

```
lm(formula = mcs ~ age + female + cesd, data = helpdata)
```

Residuals:

Min	1Q	Median	3Q	Max
-28.217	-6.563	-0.411	5.935	32.303

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	51.68646	2.38430	21.678	<2e-16 ***
age	0.08368	0.05740	1.458	0.146
female	0.05482	1.05795	0.052	0.959
cesd	-0.70039	0.03591	-19.501	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.4 on 449 degrees of freedom

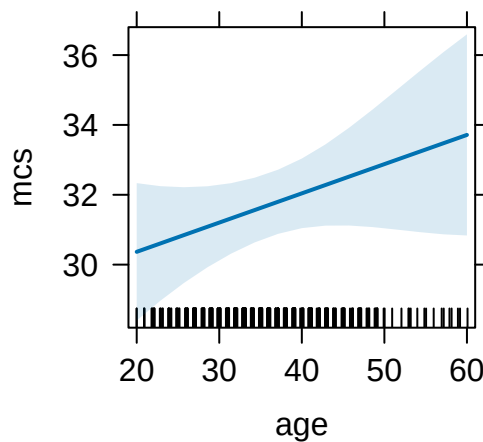
Multiple R-squared: 0.4676, Adjusted R-squared: 0.464

F-statistic: 131.4 on 3 and 449 DF, p-value: < 2.2e-16

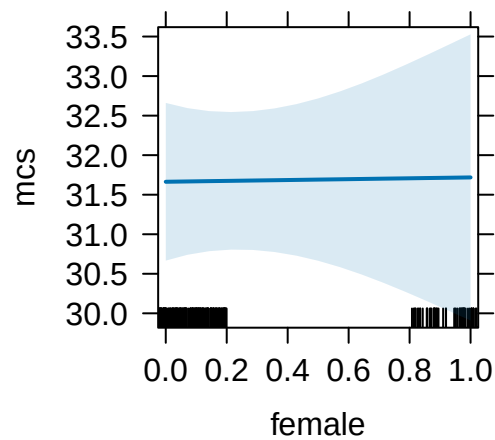
Similar to what we did above, let's use the **effects** package again to plot the effects (e.g. visualize the slopes or effects of each coefficient in the model).

```
# also look at model effects for this lm model
# library(effects)
alleff <- allEffects(fit.lmreg)
# make the effects plot
plot(allEffects(fit.lmreg))
```

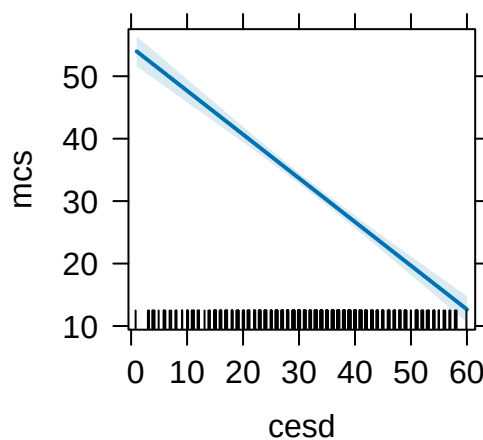
age effect plot



female effect plot



cesd effect plot



We can make a nicer formatted table using `gtsummary::tbl_regression()`. I have added some additional options to this table:

- I added `intercept=TRUE` to include the intercept
- I added `estimate_fun = ~ style_number(.x, digits = 3)` to format all of the p-values for each coefficient to 3 decimal places
- and I included some custom fit statistics using `add_glance_table()`.

```
# make formatted table
# include intercept
# format digits to 3 decimals for betas
fit.lmreg %>%
  tbl_regression(
    intercept = TRUE,
    estimate_fun = ~ style_number(.x, digits = 3)
  ) %>%
  add_glance_table(
    include = c(r.squared,
                adj.r.squared,
                p.value)
  )
```

Table 7

Characteristic	Beta	95% CI	p-value
(Intercept)	51.686	47.001, 56.372	<0.001
age	0.084	-0.029, 0.196	0.15
female	0.055	-2.024, 2.134	>0.9
cesd	-0.700	-0.771, -0.630	<0.001
R ²	0.468		
Adjusted R ²	0.464		
p-value	<0.001		

Abbreviation: CI = Confidence Interval

Side-by-side Model Comparisons

Another nice package for displaying linear model results, is the [modelsummary package](#). It is especially useful for comparing models side-by-side. For example, let's run two models and compare them:

- MODEL 1: `mcs ~ cesd` (*direct association*)
- MODEL 2: `mcs ~ cesd + age + female` (*association between `cesd` and `mcs` adjusted for `age` and gender `female`*)

Before you can perform the side-by-side comparison, we have to define the models we want to compare by creating a `list` object that essentially has the 2 model functions defined.

```
# modelsummary package alternative to gtsummary =====  
library(modelsummary)  
models <- list(  
  "main" = lm(mcs ~ cesd, data=helpdata),  
  "adj" = lm(mcs ~ age + female + cesd, data=helpdata)  
)
```

Then we can display the results using `modelsummary()`. There are many options we can add but for now I'm asking to display the confidence intervals for each coefficient.

```
# show side-by-side model results, add confidence intervals  
modelsummary(models, statistic = 'conf.int')
```

Table 8

	main	adj
(Intercept)	54.658	51.686
	[52.217, 57.098]	[47.001, 56.372]
cesd	-0.700	-0.700
	[-0.769, -0.630]	[-0.771, -0.630]
age		0.084
		[-0.029, 0.196]
female		0.055
		[-2.024, 2.134]
Num.Obs.	453	453
R2	0.465	0.468
R2 Adj.	0.464	0.464
AIC	3319.8	3321.6
BIC	3332.1	3342.2
Log.Lik.	-1656.886	-1655.809
F	392.023	131.427
RMSE	9.38	9.36

These results also provide for the R2 of each model. As we can see the R2 for the MAIN model was `r2=.465` and for the ADJ model was `r2=.468`, so the difference for adding `age` and `female` to the model was a change in `r2=.003`.

We can get a formal test of the differences in these two model by using the `anova()` function in the `stats` package.

```
# formal test of model comparisons
main <- lm(mcs ~ cesd, data=helpdata)
adj <- lm(mcs ~ age + female + cesd, data=helpdata)
anova(main, adj)
```

Analysis of Variance Table

Model 1: mcs ~ cesd

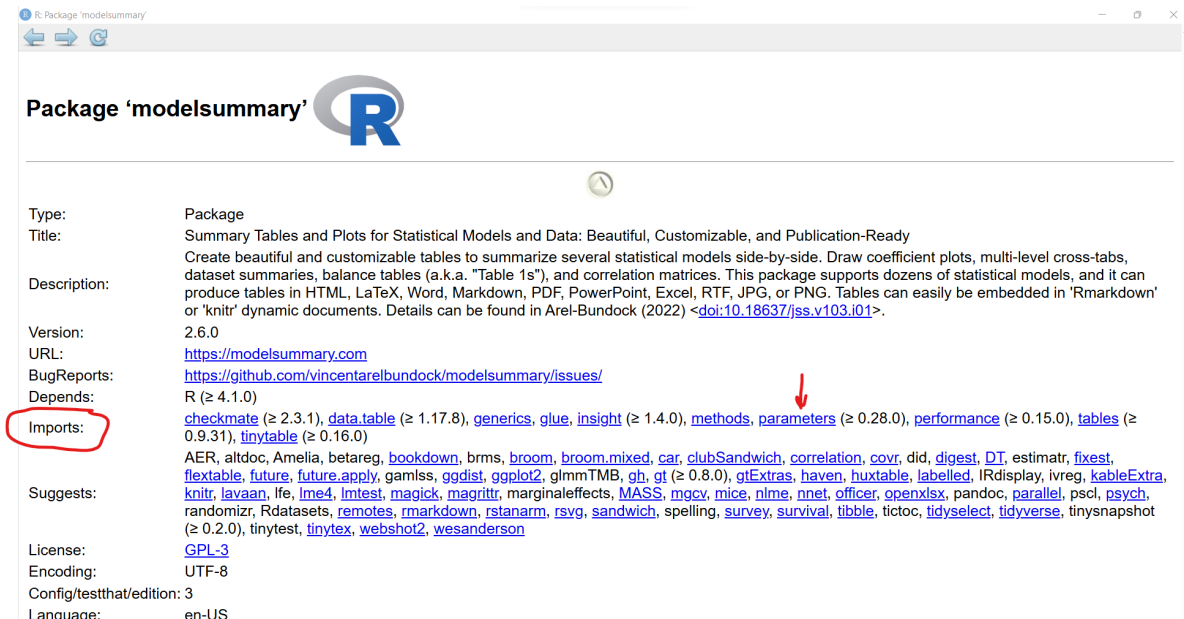
Model 2: mcs ~ age + female + cesd

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	451	39862				
2	449	39673	2	188.96	1.0693	0.3441

The change in r^2 between these two models was small and as we can see the F-test for these differences was not statistically significant, $p=.3441$.

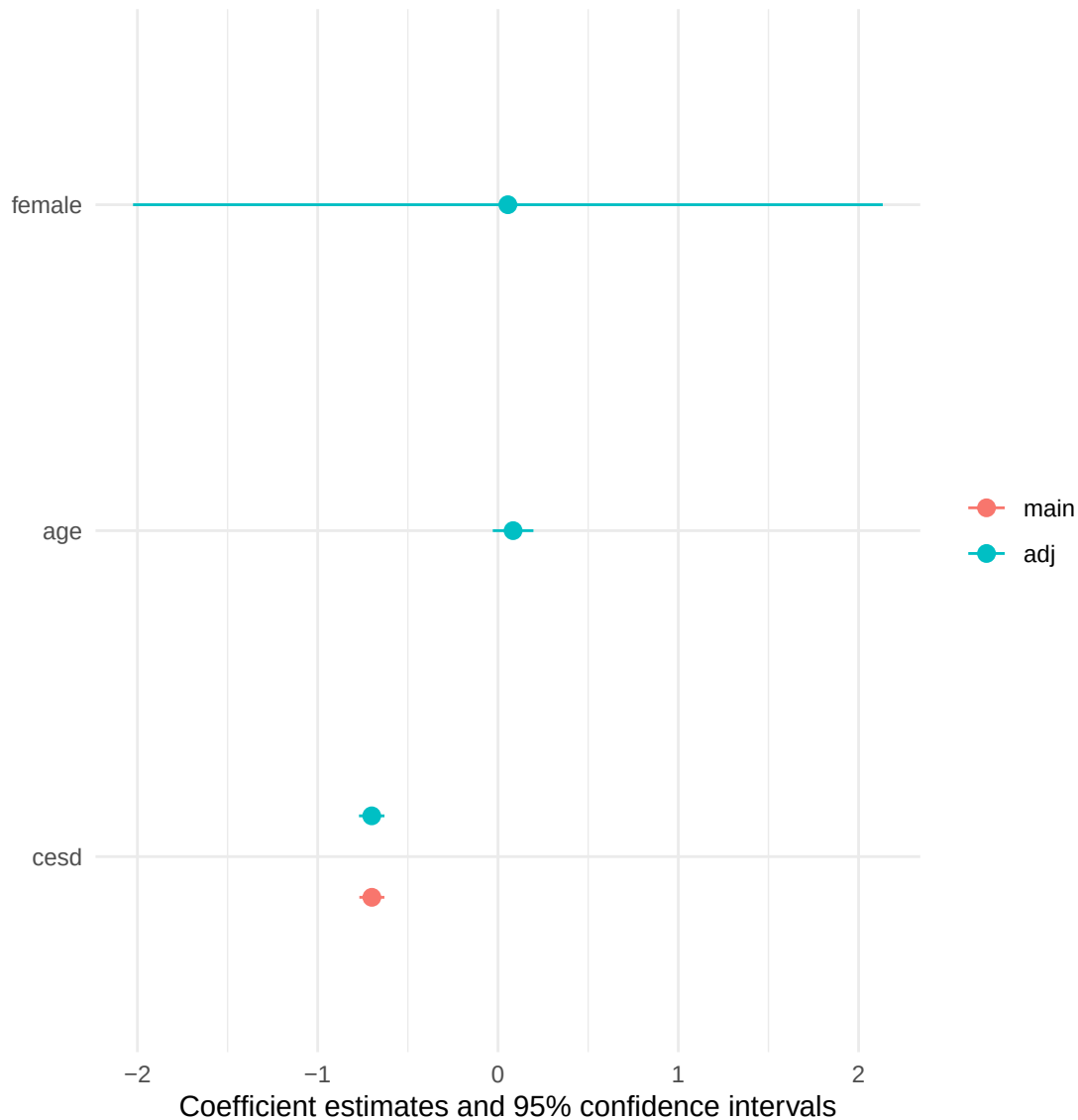
I also like seeing the standardized regression coefficients which are also effect sizes useful for interpreting the weight of each coefficient in the model without being affected by different units or scales of the different variables. The [modelsummary package](#) also leverages features from the [parameters package](#), which is part of the [easystats](#) suite of packages..

We can see more details about the other packages that get installed and loaded along with `modelsummary` by looking at the DESCRIPTION for the `modelsummary` package in the “Help” window and look at the list of packages mentioned in “Imports:”



The `modelsummary` package also has a function `modelplot()` that creates a visual plot of each model coefficient estimate along with their 95% confidence intervals. This can be helpful for comparing the estimates from two different models (such as `main` versus `adj`):

```
# plot comparisons of model coefficients,  
# remove intercept coefficient from plot  
modelplot(models, coef_omit = "(Intercept)")
```



We can add `standardize = "refit"` to `modelsummary()` to get the standardized coefficients in our side-by-side table. Learn more with `help(standardize_parameters, package = "parameters")`.

```
# also get the standardized coefficients
modelsummary(models,
  standardize = "refit",
  statistic = 'conf.int')
```

Table 9

	main	adj
(Intercept)	0.000	0.000
	$[-0.068, 0.068]$	$[-0.068, 0.068]$
cesd	-0.682	-0.683
	$[-0.750, -0.614]$	$[-0.751, -0.614]$
age		0.050
		$[-0.017, 0.118]$
female		0.002
		$[-0.067, 0.071]$
Num.Obs.	453	453
R2	0.465	0.468
R2 Adj.	0.464	0.464
AIC	3319.8	3321.6
BIC	3332.1	3342.2
Log.Lik.	-1656.886	-1655.809
F	392.023	131.427
RMSE	9.38	9.36

OPTIONAL- `olsrr` package

Another really good package for diving into everything for an “ordinary least squares” regression model, is the [olsrr package](#). This package has a number of really good functions for displaying results, evaluating model fit diagnostics and tests and running variable selection procedures and much more we won’t have time to get into. But here is a glimpse of what you can do to get the model results, evaluate multicollinearity statistics, plot model diagnostics and test for the normality of the residuals.

```
# olsrr::ols_regress gives a nice summary
# including the standardized regression coefficients
library(olsrr)
olsrr::ols_regress(fit.lmreg)
```

Model Summary

R	0.684	RMSE	9.358
R-Squared	0.468	MSE	87.579
Adj. R-Squared	0.464	Coef. Var	29.675
Pred R-Squared	0.459	AIC	3321.619
MAE	7.398	SBC	3342.198

RMSE: Root Mean Square Error

MSE: Mean Square Error

MAE: Mean Absolute Error

AIC: Akaike Information Criteria

SBC: Schwarz Bayesian Criteria

ANOVA

	Sum of Squares	DF	Mean Square	F	Sig.
Regression	34838.341	3	11612.780	131.427	0.0000
Residual	39673.218	449	88.359		
Total	74511.559	452			

Parameter Estimates

model	Beta	Std. Error	Std. Beta	t	Sig	lower	upper
-------	------	------------	-----------	---	-----	-------	-------

(Intercept)	51.686	2.384		21.678	0.000	47.001	56.372
age	0.084	0.057	0.050	1.458	0.146	-0.029	0.196
female	0.055	1.058	0.002	0.052	0.959	-2.024	2.134
cesd	-0.700	0.036	-0.683	-19.501	0.000	-0.771	-0.630

```
# olsrr also has good model fit checks
# like multicollinearity checks,
# VIF and condition index
# get VIF and multicollinearity stats
ols_coll_diag(fit.lmreg)
```

Tolerance and Variance Inflation Factor

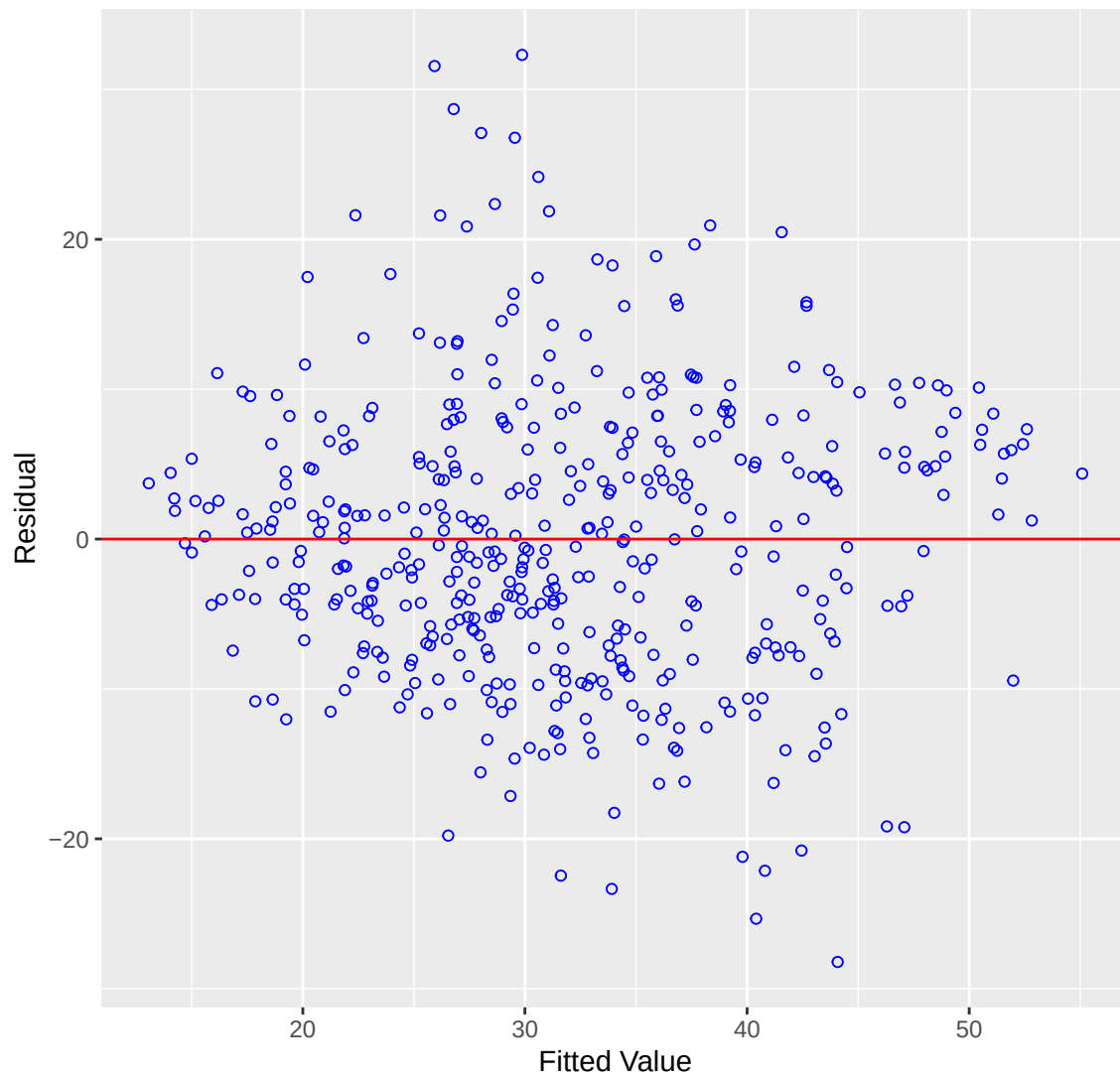
```
-----
Variables Tolerance      VIF
1      age 0.9981296 1.001874
2    female 0.9659547 1.035245
3     cesd 0.9676967 1.033382
```

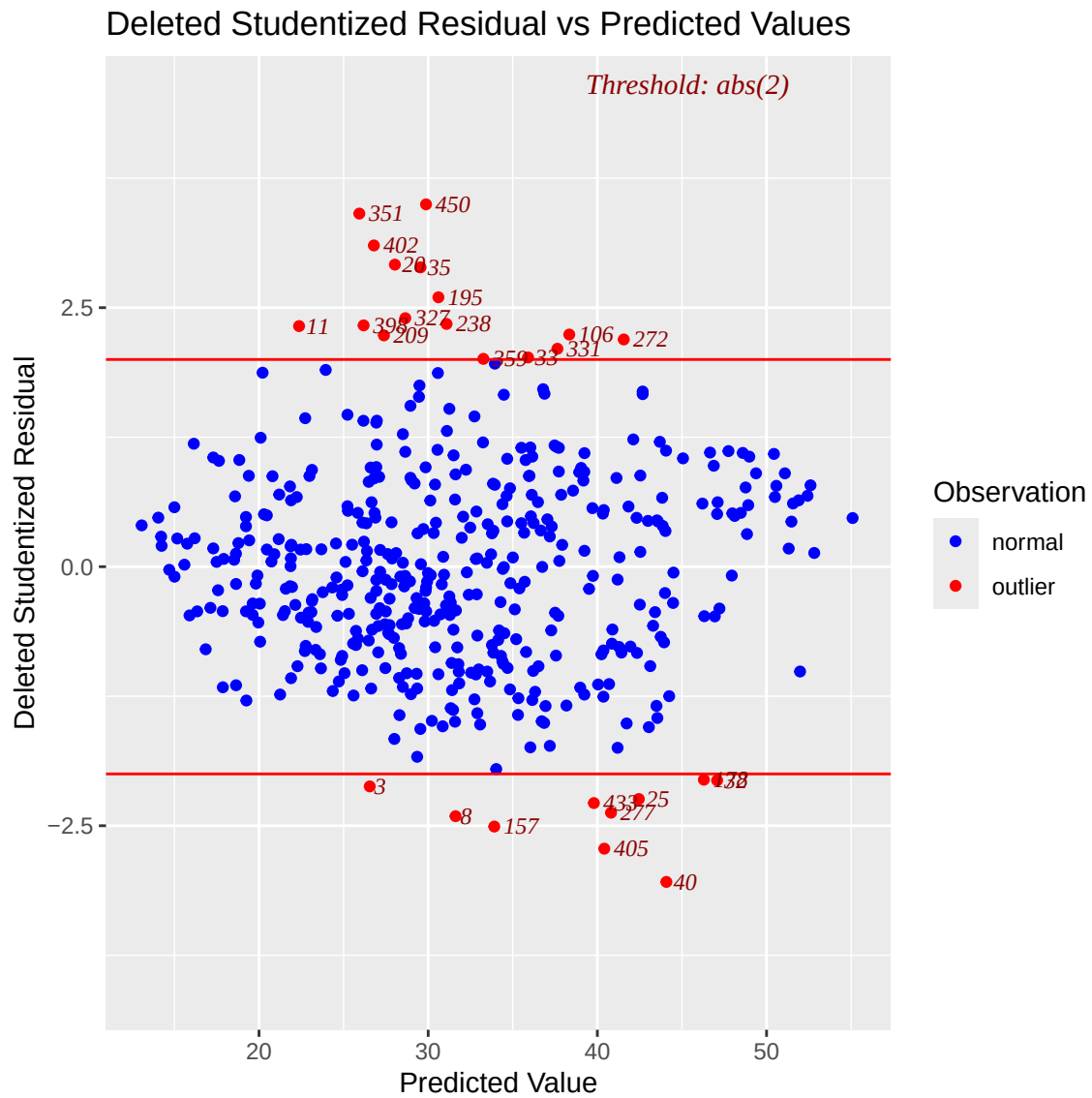
Eigenvalue and Condition Index

```
-----
Eigenvalue Condition Index  intercept      age      female      cesd
1 3.21370763      1.000000 0.003143355 0.004020483 0.029731756 0.01088393
2 0.67195014      2.186929 0.002653217 0.003411204 0.953661822 0.00509675
3 0.09380034      5.853302 0.027347222 0.121454787 0.014875201 0.87094676
4 0.02054189     12.507861 0.966856206 0.871113527 0.001731221 0.11307256
```

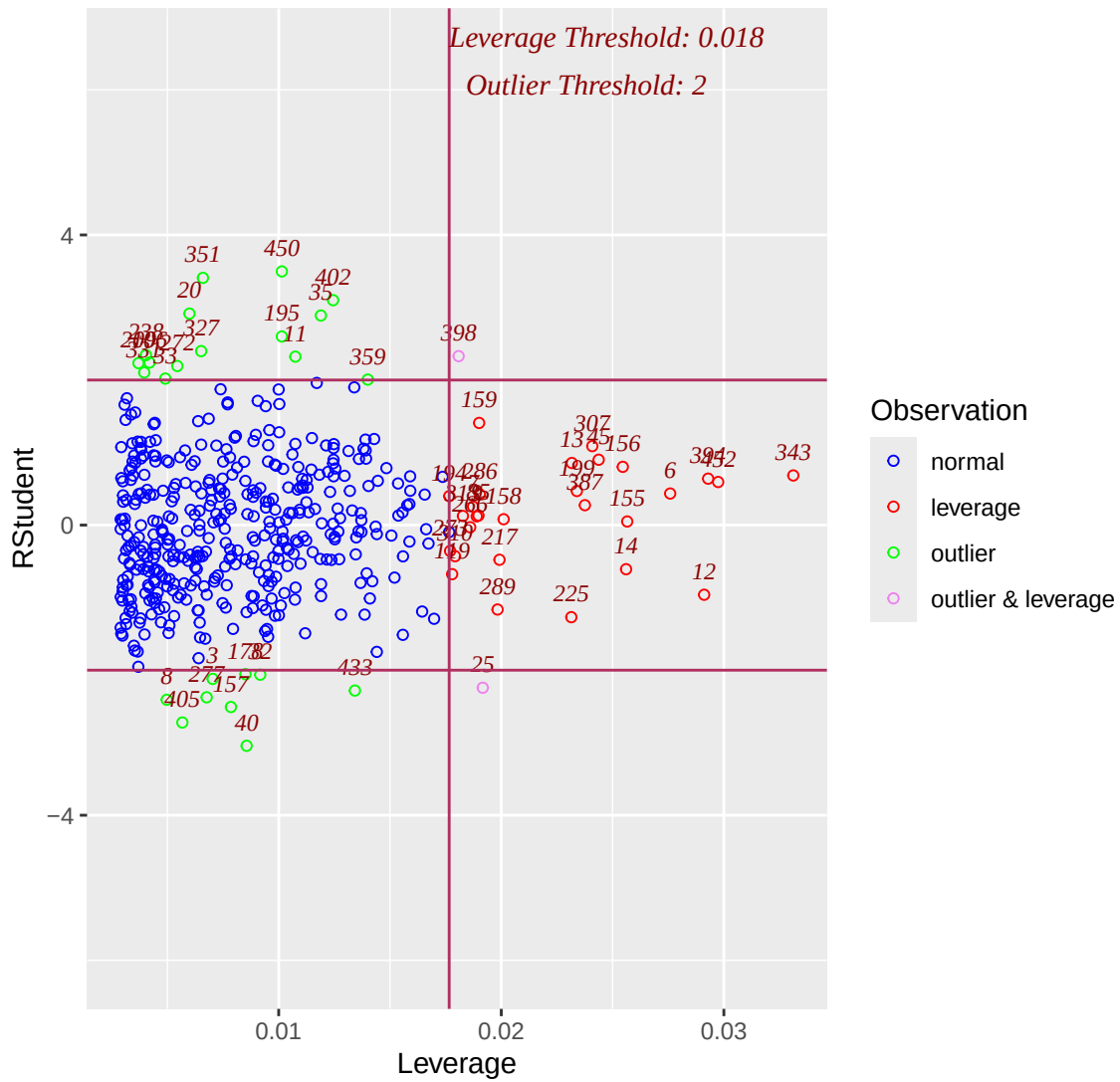
```
# diagnostic plots  
# check for new output windows  
ols_plot_diagnostics(fit.lmreg)
```

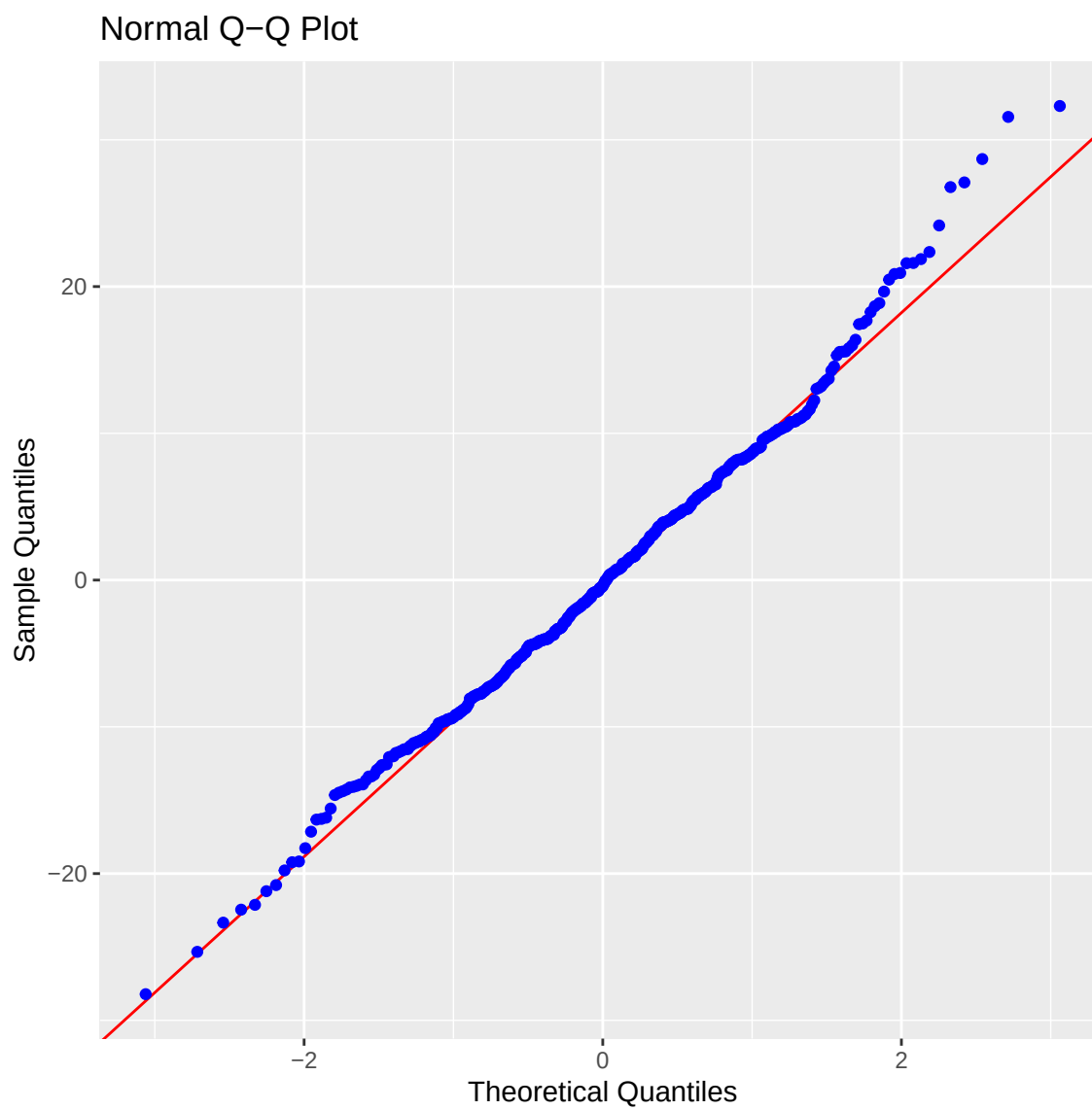
Residual vs Fitted Values

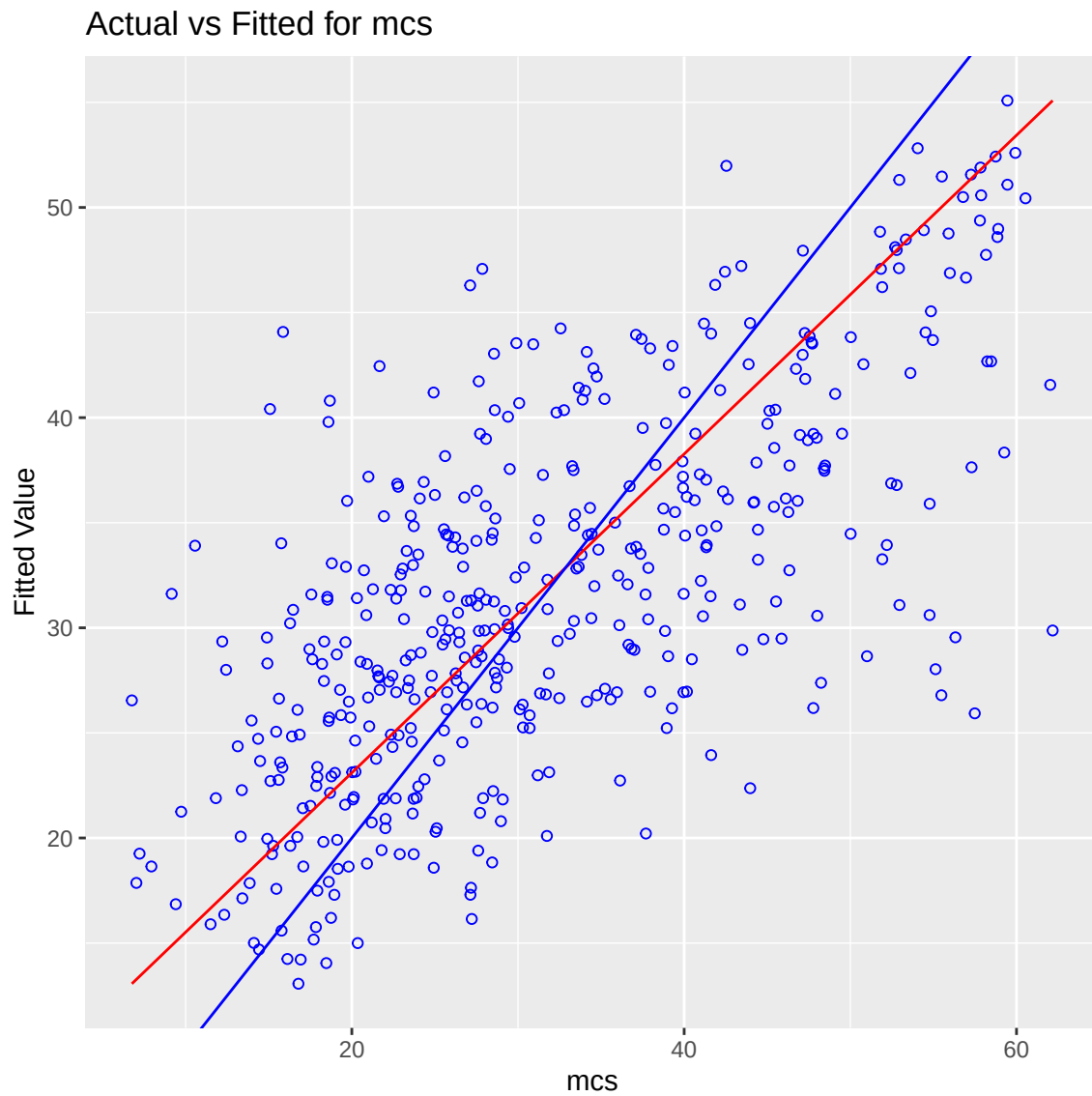


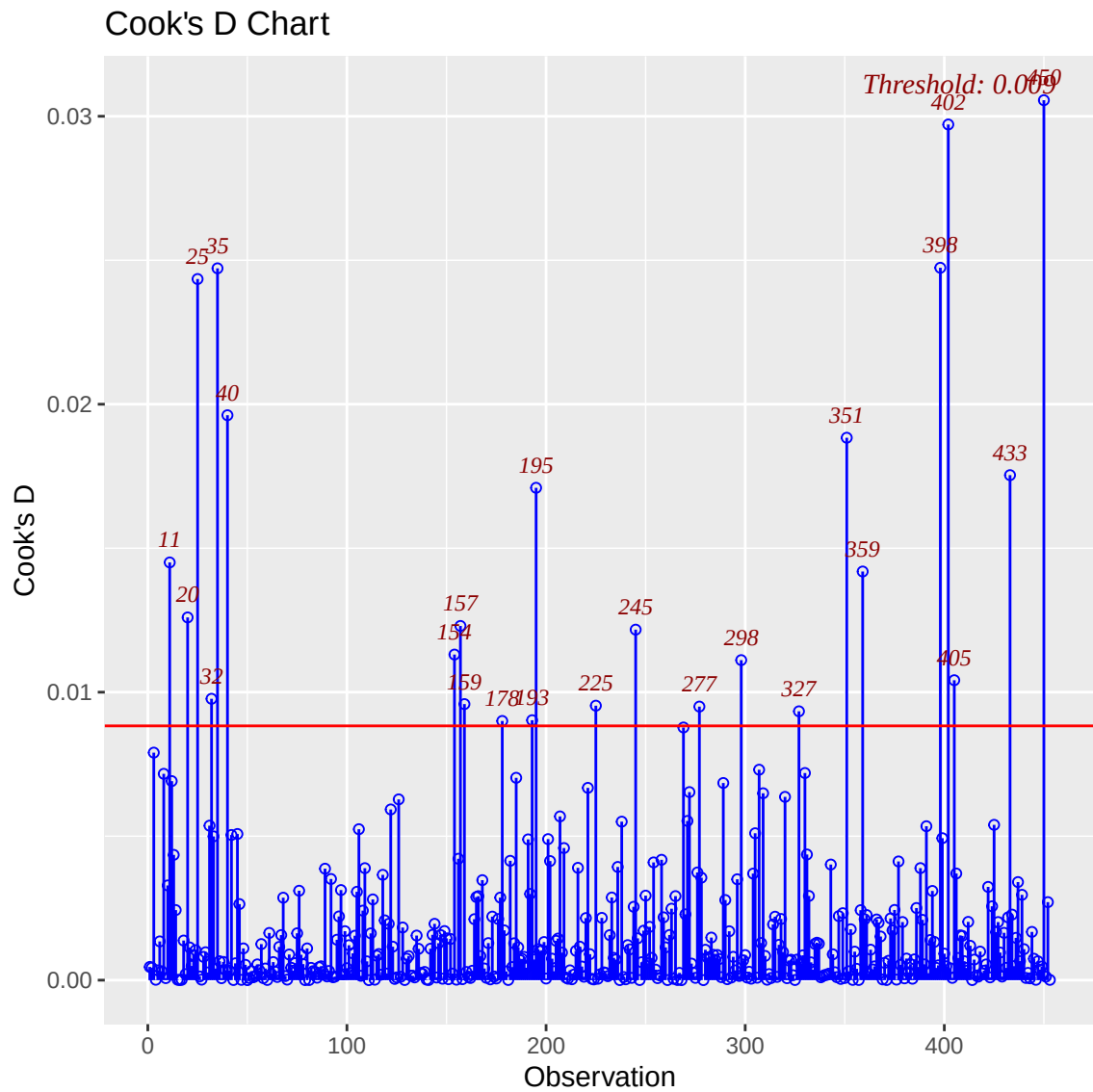


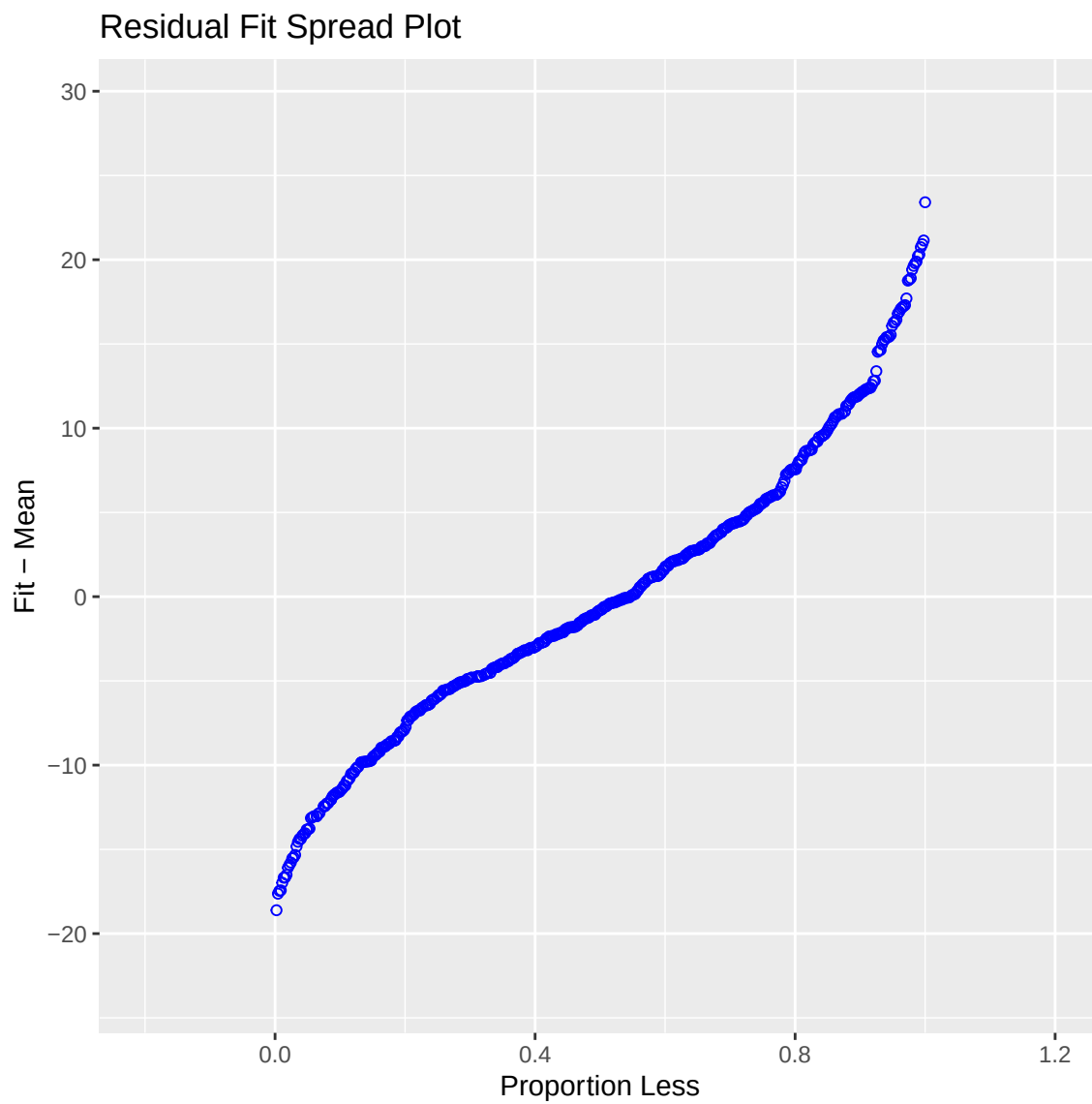
Outlier and Leverage Diagnostics for mcs

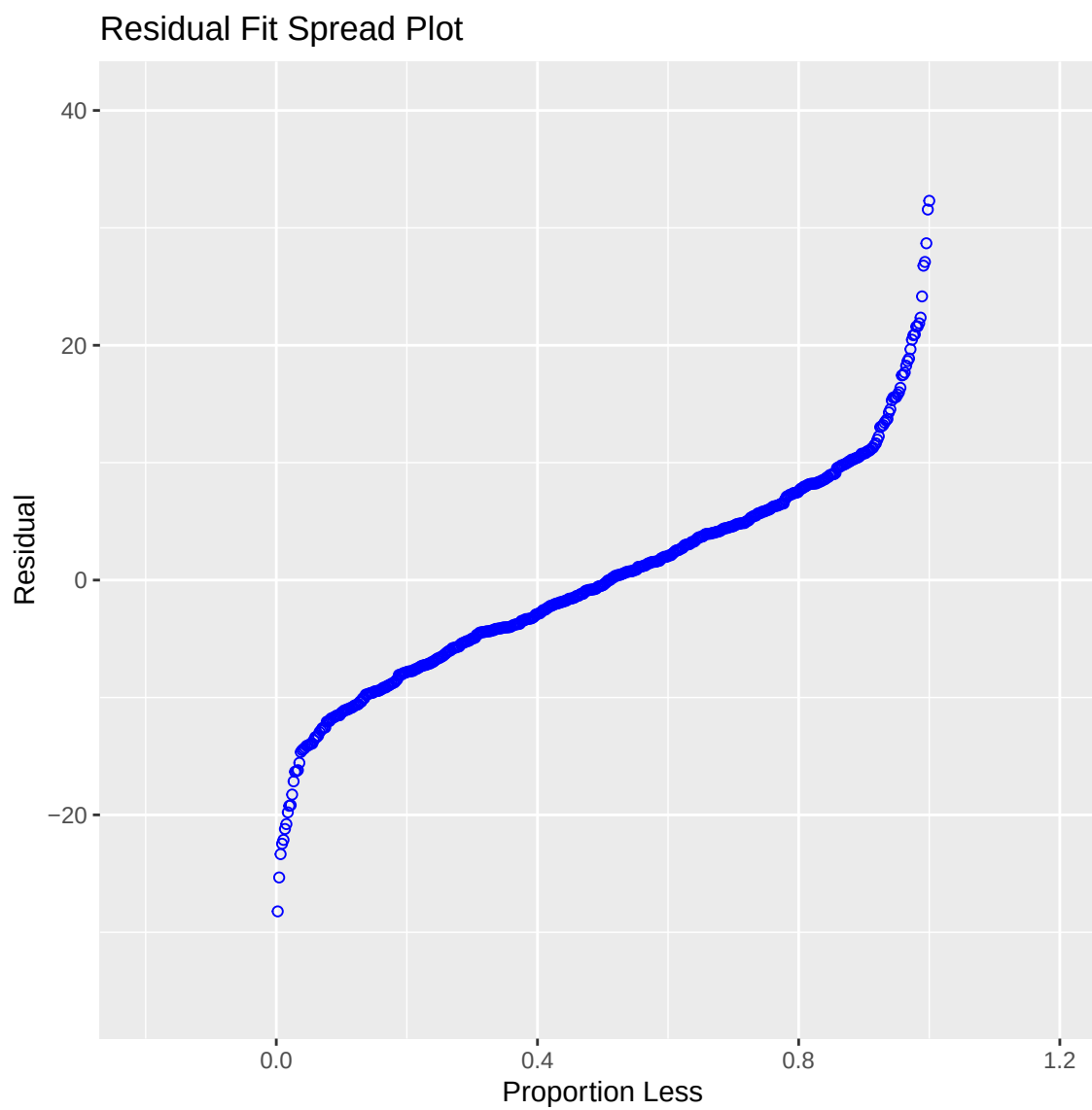


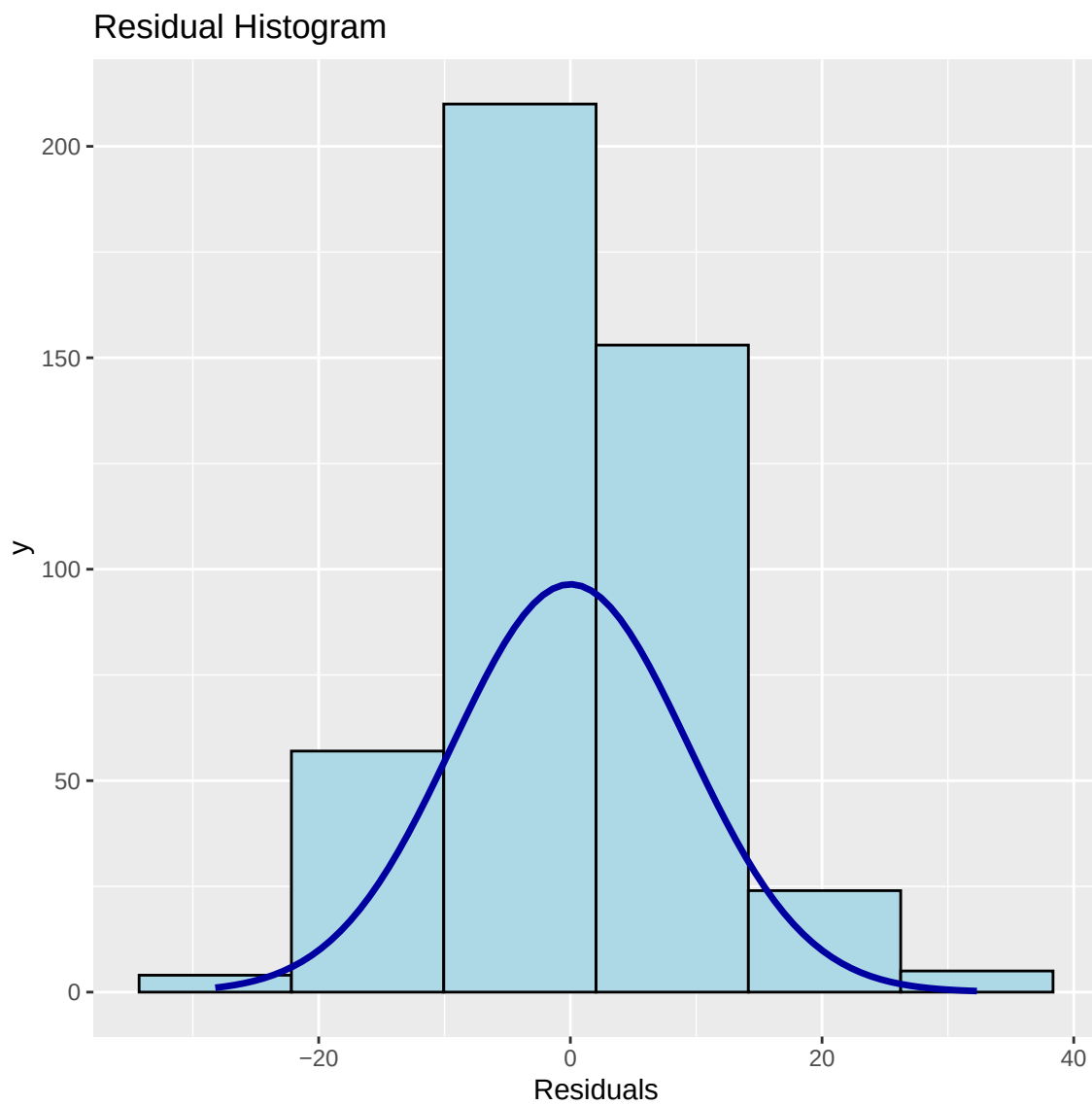


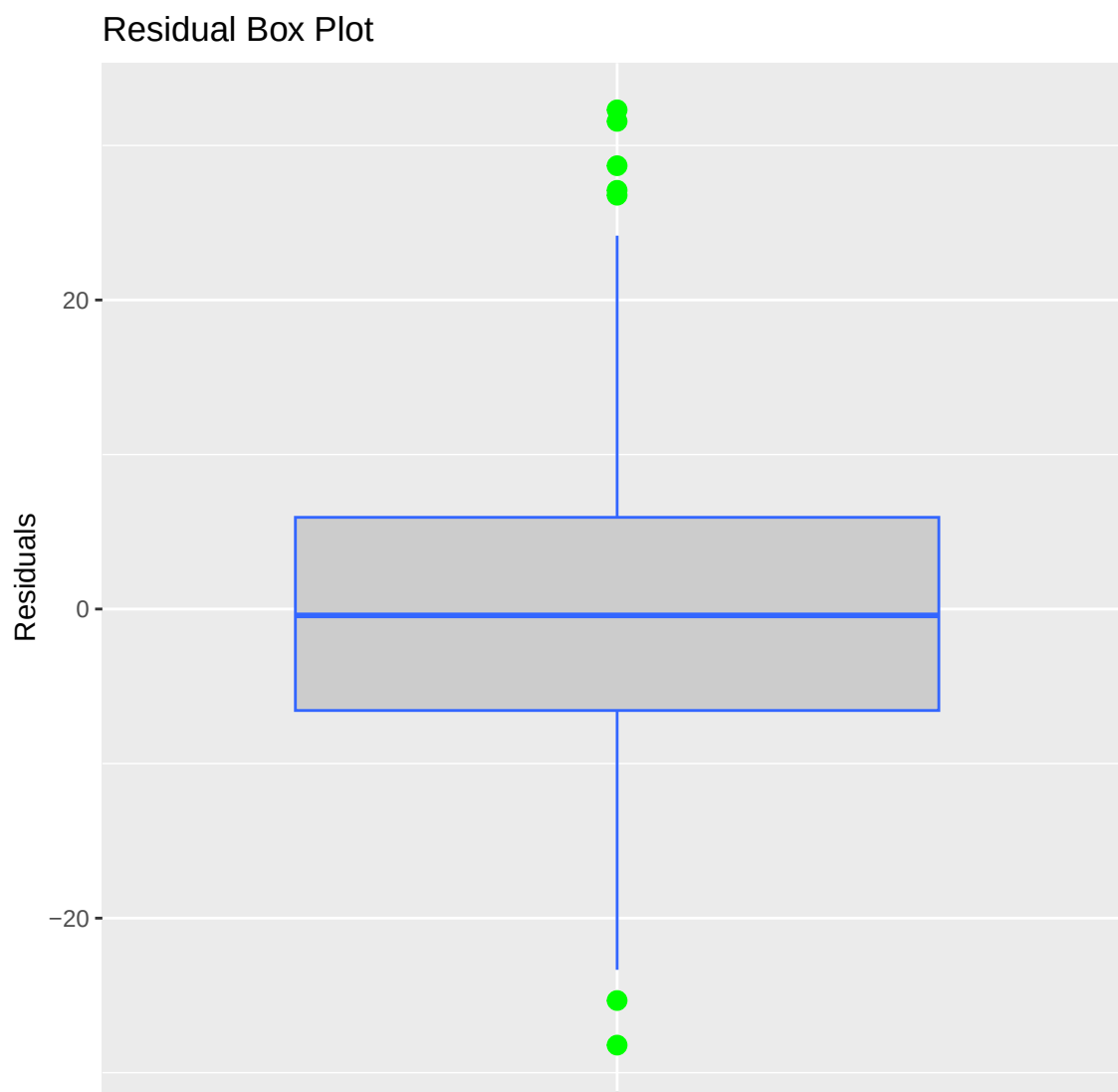




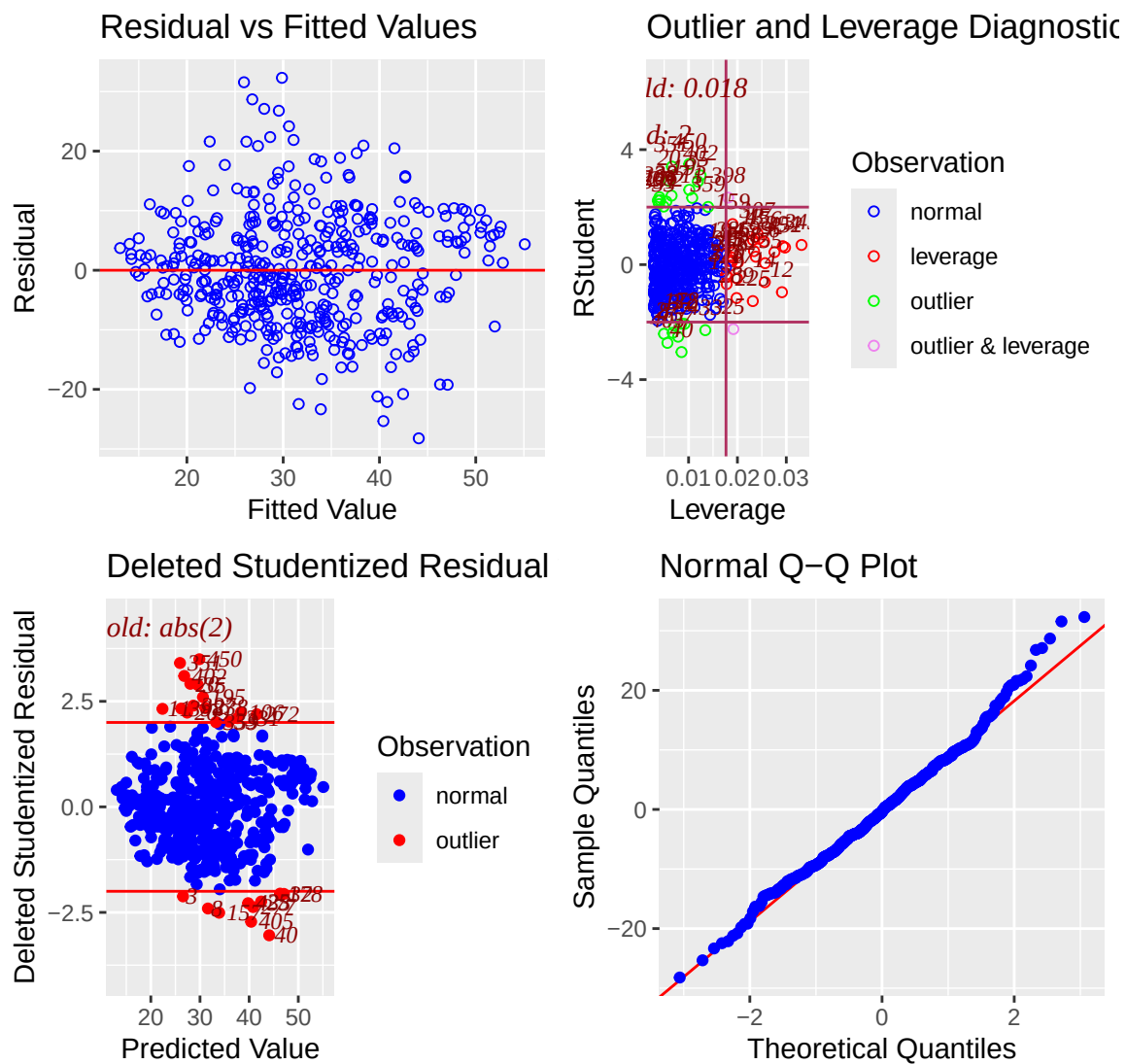






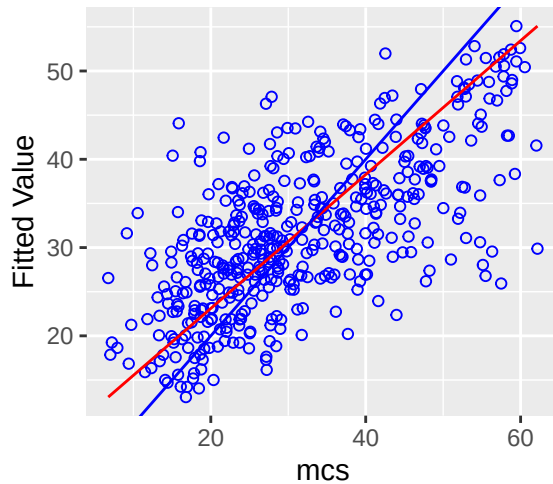


Regression Diagnostics

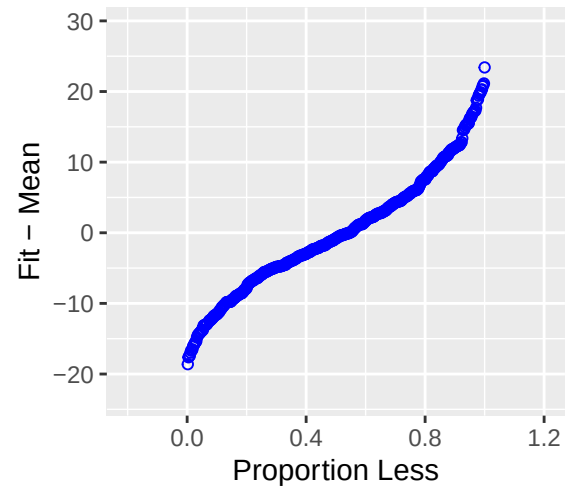


Regression Diagnostics

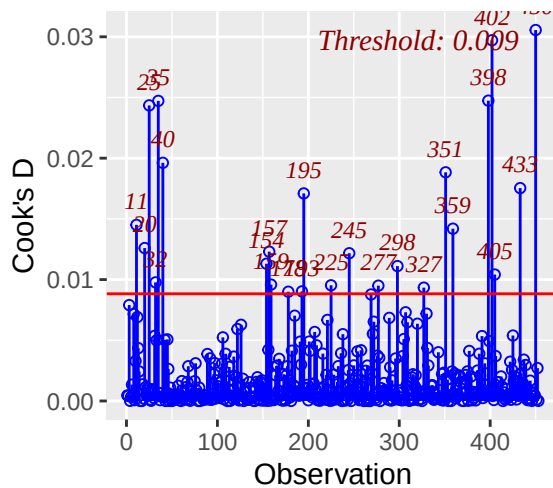
Actual vs Fitted for mcs



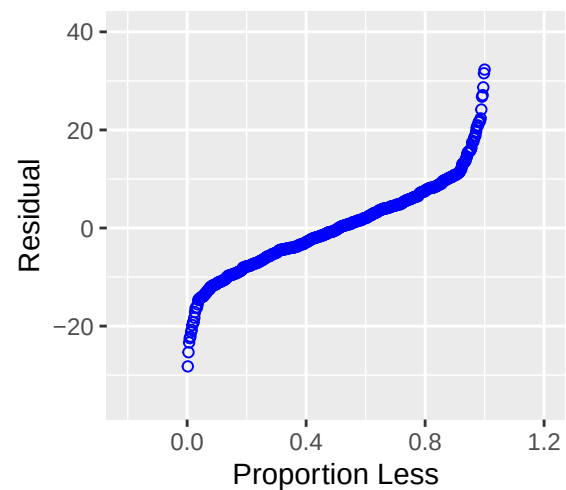
Residual Fit Spread Plot



Cook's D Chart

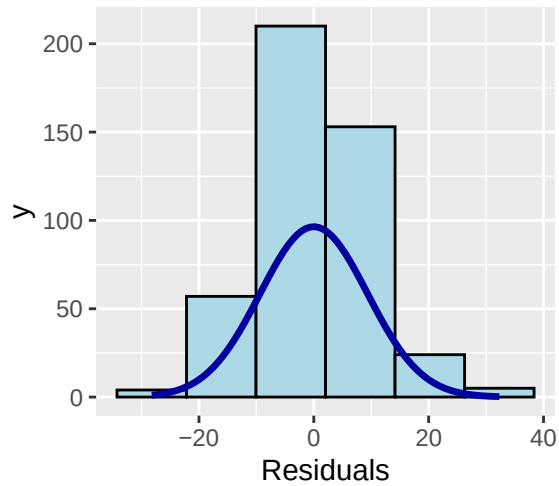


Residual Fit Spread Plot

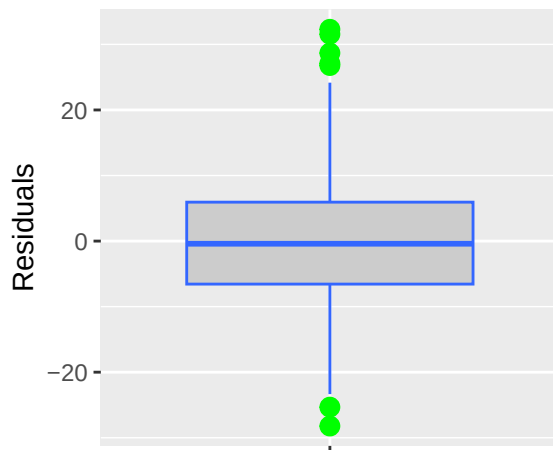


Regression Diagnostics

Residual Histogram



Residual Box Plot



```
# normality tests for residuals  
ols_test_normality(fit.lmreg)
```

Test	Statistic	pvalue
Shapiro-Wilk	0.9922	0.0184
Kolmogorov-Smirnov	0.0282	0.8634
Cramer-von Mises	34.5818	0.0000
Anderson-Darling	0.6588	0.0850

SAS Code for Linear Regression

```
* Regression;

proc reg data=help;
  model mcs = age female cesd / STB CLB;
  run;

* Compare "main" vs "adj" models
  "main" for mcs = cesd
  "adj"  for mcs = cesd + age + female;

proc reg data=help
  plots=none outest=rsq_data rsquare;
  var mcs cesd age female;
  model mcs=cesd;
  run ;
  model mcs=cesd age female;
  run ;
quit;

* get change in r2 between the 2 models;

data rsq_extended;
  set rsq_data;
  delta_rsq=dif(_rsq_);
  delta_indvars=dif(_in_);
run;

proc print data=rsq_extended; run;

* Test coefficients for age and female = 0
  which is a test of covariates
  significant or not, "main" vs "adj";

proc reg data=help;
  model mcs=cesd age female;
  test age=0, female=0;
  run;
quit;
```

G. Logistic Regression

Moving from linear to logistic regression is very straightforward. Instead of using the `lm()` function, we will use the `glm()` function to fit “generalized linear models”. In fact, the `glm()` function can be used to perform simple linear regression as well. More about that below.

Since we’ve been looking at a regression model for the `mcs` at baseline, let’s dichotomize this outcome into MCS scores ≤ 50 and MCS scores > 50 . The SF36 `mcs` and `pcs` scores have been population normalized into “T-scores” that have been re-scaled into scales from 0-100 with a population mean of 50 and population standard deviation of 10. So, individuals who have MCS scores > 50 are considered to have mental health quality of life scores better than the population average.

I added a `table()` of the split results - as you can see there are only 52/435 (11.5%) of these individuals in the HELP dataset that had mental quality of life scores better than the population average. This makes sense since the HELP participants include a high proportion of homeless and people with unstable housing. So, keep this in mind when reviewing the logistic regression results.

```
# split score mcs > 50
helpdata <- helpdata %>%
  mutate(mcs_gt50 = as.numeric(mcs > 50))

table(helpdata$mcs_gt50)
```

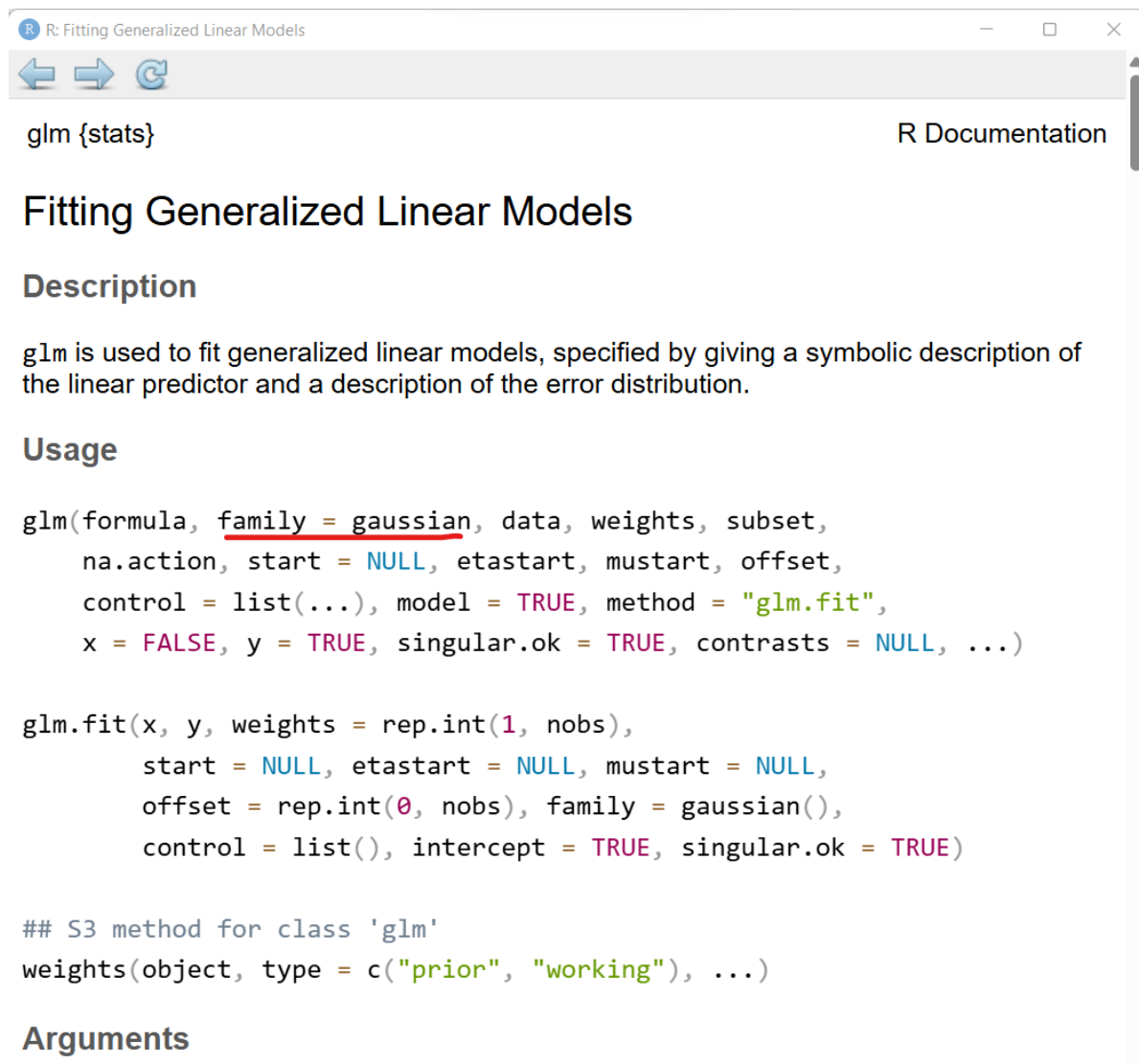
```
  0    1
401  52
```

To fit a logistic regression model using `glm()`, the syntax is nearly identical to `lm()`, except we MUST define the `family = xxxxx` argument.

See `help(glm, package="stats")`.

! `family=gaussian` default setting for `glm()`

Notice that `family=gaussian` is the default which is the setting for simple linear regression!. So, if you forget to set `family=binomial` you'll end up running a linear regression instead of a logistic regression.



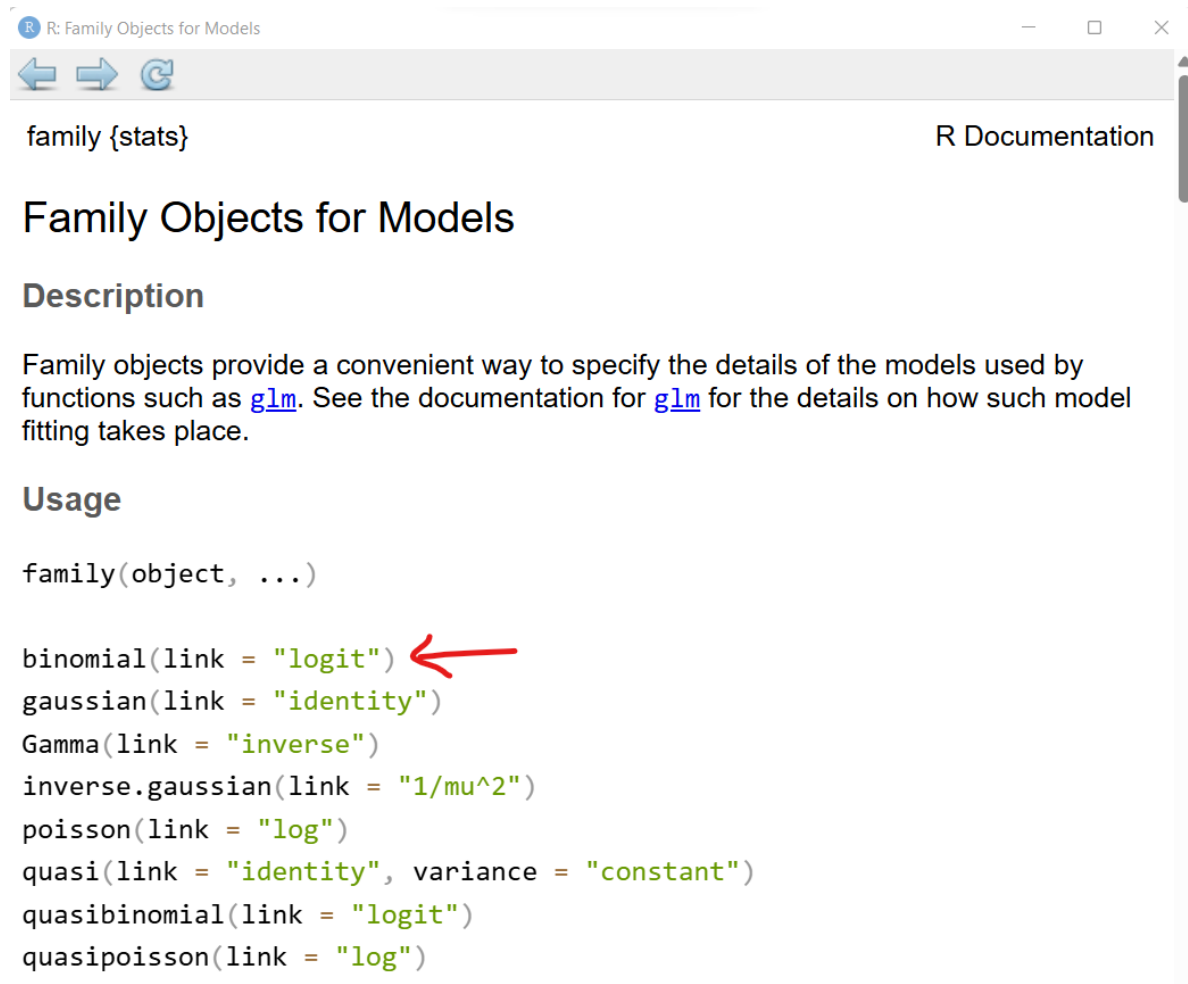
The screenshot shows the R documentation page for the `glm` function. The title is "Fitting Generalized Linear Models". The section "Description" states that `glm` is used to fit generalized linear models, specified by giving a symbolic description of the linear predictor and a description of the error distribution. The "Usage" section shows the syntax for `glm()` and `glm.fit()`. The `glm()` function has arguments: `formula`, `family = gaussian`, `data`, `weights`, `subset`, `na.action`, `start = NULL`, `etastart`, `mustart`, `offset`, `control = list(...)`, `model = TRUE`, `method = "glm.fit"`, `x = FALSE`, `y = TRUE`, `singular.ok = TRUE`, `contrasts = NULL`, and `...`. The `glm.fit()` function has arguments: `x`, `y`, `weights = rep.int(1, nobs)`, `start = NULL`, `etastart = NULL`, `mustart = NULL`, `offset = rep.int(0, nobs)`, `family = gaussian()`, `control = list()`, `intercept = TRUE`, `singular.ok = TRUE`, and `...`. There is also an S3 method for class 'glm' with arguments `object`, `type = c("prior", "working")`, and `...`.

```
glm(formula, family = gaussian, data, weights, subset,
    na.action, start = NULL, etastart, mustart, offset,
    control = list(...), model = TRUE, method = "glm.fit",
    x = FALSE, y = TRUE, singular.ok = TRUE, contrasts = NULL, ...)
```

```
glm.fit(x, y, weights = rep.int(1, nobs),
        start = NULL, etastart = NULL, mustart = NULL,
        offset = rep.int(0, nobs), family = gaussian(),
        control = list(), intercept = TRUE, singular.ok = TRUE)
```

```
## S3 method for class 'glm'
weights(object, type = c("prior", "working"), ...)
```

Scroll down to “Arguments” and click on a link for the details on “family” - notice there are many options besides binomial with the logit link function. If you want to run a simple linear regression, just keep the default `family=gaussian`. You can also run a Poisson regression for count data or use Gamma distributions for time-to-event data.



```
family {stats} R Documentation
```

Family Objects for Models

Description

Family objects provide a convenient way to specify the details of the models used by functions such as `glm`. See the documentation for `glm` for the details on how such model fitting takes place.

Usage

```
family(object, ...)
```

```
binomial(link = "logit")  
gaussian(link = "identity")  
Gamma(link = "inverse")  
inverse.gaussian(link = "1/mu^2")  
poisson(link = "log")  
quasi(link = "identity", variance = "constant")  
quasibinomial(link = "logit")  
quasipoisson(link = "log")
```

So, when we want to run a logistic regression, we need to choose `family = binomial` to get the logit link function for the dichotomous outcome `mcs_gt50`.

```
# fit a logistic regression model
# use glm instead of lm
# and add family = binomial
fit.logreg <-
  glm(mcs_gt50 ~ age + female + cesd,
      family = binomial,
      data = helpdata)
```

See the model results

```
summary(fit.logreg)
```

Call:

```
glm(formula = mcs_gt50 ~ age + female + cesd, family = binomial,
    data = helpdata)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.33489	0.99900	1.336	0.181
age	0.01238	0.02461	0.503	0.615
female	-0.01291	0.46934	-0.028	0.978
cesd	-0.15086	0.01876	-8.042	8.84e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 322.91 on 452 degrees of freedom
Residual deviance: 213.27 on 449 degrees of freedom
AIC: 221.27

Number of Fisher Scoring iterations: 6

These model coefficients (Estimate's) do not look like odds ratios. Why not?

The linear logistic-regression or the linear logit model is given by this equation:

$$\pi_i = \frac{1}{1 + \exp[-(\alpha + \beta X_i)]}$$

where π_i is the probability of the desired outcome.

Given this equation then

$$Odds = \frac{\pi}{1 - \pi}$$

and

$$Logit = \log_e\left(\frac{\pi}{1 - \pi}\right)$$

and

$$Logit = \alpha + \beta X_i$$

So, to get the odds ratios, we have to compute the exponential function of the coefficients using the `exp()` function for the coefficients which you can get via `coef(fit.logreg)`:

```
# to see raw coefficients  
coef(fit.logreg)
```

```
(Intercept)      age      female      cesd  
  1.33489444  0.01238475 -0.01291043 -0.15086233
```

```
# exponentiate the coefficients  
#to get odds ratios  
exp(coef(fit.logreg))
```

```
(Intercept)      age      female      cesd  
  3.7995948  1.0124618  0.9871726  0.8599661
```

We'll make a better of these tables in a moment. but first let's take a look at what model fit statistics are available via the **broom** package.

```
# model statistics available
library(broom)
bg <- broom::glance(fit.logreg)
bg
```

```
# A tibble: 1 x 8
  null.deviance df.null logLik   AIC   BIC deviance df.residual  nobs
      <dbl>    <int>  <dbl> <dbl> <dbl>   <dbl>      <int> <int>
1      323.     452  -107.  221.  238.    213.       449   453
```

```
names(bg)
```

```
[1] "null.deviance" "df.null"      "logLik"      "AIC"
[5] "BIC"           "deviance"     "df.residual" "nobs"
```

Let's use `usgetsummary::tbl_regression` with `exponentiate = TRUE` to get a nicer table of logistic regression model results with the odds ratios (OR's) and the confidence intervals for the OR's.

```
# make formatted table
# include intercept
# remember to exponentiate the coefficients
# format digits to 3 decimals for betas
# and do 3 decimal digits for p-values
fit.logreg %>%
  tbl_regression(
    intercept = TRUE,
    exponentiate = TRUE,
    estimate_fun = ~ style_number(.x, digits = 3),
    pvalue_fun = label_style_pvalue(digits = 3)
  ) %>%
  add_glance_table(
    include = c(AIC, BIC, nobs)
  )
```

Table 10

Characteristic	OR	95% CI	p-value
(Intercept)	3.800	0.556, 28.330	0.181
age	1.012	0.964, 1.063	0.615
female	0.987	0.375, 2.403	0.978
cesd	0.860	0.827, 0.890	<0.001
AIC	221		
BIC	238		
No. Obs.	453		

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

SAS Code for Logistic Regression

```
* logistic regression;  
* compute mcs > 50;  
data help;  
  set help;  
  if not missing(mcs) then mcs_gt50 = mcs > 16;  
  else mcs = .;  
  run;  
  
proc logistic data=help plots=roc descending;  
  model mcs_gt50 = age female cesd / ctable lackfit;  
  run;
```

2. Merging Datasets

For this exercise we will use 4 datasets which were subsets of the original `helpdata` dataset.

- `dat1.Rdata`: all baseline cases, with only these variables: `id`, `age`, `cesd`
- `dat2.Rdata`: 12 month cases, with these variables: `id`, `female`, `cesd2`
- `treat1.Rdata`: only control group cases, with these variables: `id`, `treat`, `age`, `cesd`, `pcs`
- `treat2.Rdata`: only the treatment intervention cases, with these variables: `id`, `treat`, `age`, `cesd`, `pcs`

First load the datasets:

```
load(file = "help.RData")
load(file = "dat1.Rdata")
load(file = "dat2.Rdata")
load(file = "treat1.Rdata")
load(file = "treat2.Rdata")
```

R code to perform joins (adding columns)

To perform an “inner join” which keeps all cases in common between both datasets, we can use the `inner_join()` function from `dplyr`. See [Chapter 19.4 in “R for Data Science \(2e\)”](#) for more detailed explanation and helpful illustrations.

For the joins below, we are using `id` as our “primary key” to identify the cases we want to match up between both datasets.

```
# join by ids
# keep only cases with both
# cesd and cesd2
dat12_inner <-
  inner_join(x = dat1,
            y = dat2,
            by = join_by(id))
head(dat12_inner)
```

	id	age	cesd	female	cesd2
1	5	32	39	0	27
2	7	49	52	1	47
3	10	39	46	0	47
4	16	36	43	0	3
5	17	28	35	1	25
6	18	35	19	0	11

An `outer_join()` will keep ALL cases regardless of which dataset they come from.

```
# keep all cases from either dataset
dat12_full <-
  full_join(x = dat1,
            y = dat2,
            by = join_by(id))
head(dat12_full)
```

	id	age	cesd	female	cesd2
1	1	37	49	NA	NA
2	2	37	30	NA	NA
3	3	26	39	NA	NA
4	4	39	15	NA	NA
5	5	32	39	0	27
6	6	47	6	NA	NA

If we have a primary dataset that we want to define which cases we want to keep, we can perform a `left_join()` to keep all cases in the 1st dataset and only add cases from the 2nd dataset that match.

```
# keep only cases from left (first)
# dataset
dat12_left <-
  left_join(x = dat1,
            y = dat2,
            by = join_by(id))
head(dat12_left)
```

	id	age	cesd	female	cesd2
1	1	37	49	NA	NA
2	2	37	30	NA	NA
3	3	26	39	NA	NA
4	4	39	15	NA	NA
5	5	32	39	0	27
6	6	47	6	NA	NA

SAS code to perform joins (add columns)

For the code below, we will need to have these 4 SAS datasets:

- `dat1.sas`: all baseline cases, with only these variables: `id`, `age`, `cesd`
- `dat2.sas`: 12 month cases, with these variables: `id`, `female`, `cesd2`
- `treat1.sas`: only control group cases, with these variables: `id`, `treat`, `age`, `cesd`, `pcs`
- `treat2.sas`: only the treatment intervention cases, with these variables: `id`, `treat`, `age`, `cesd`, `pcs`

To perform an INNER JOIN in SAS using PROC SQL:

```
* perform inner join;
proc sql;
  create table dat12_inner as
  select * from rforsas.dat1 as x join rforsas.dat2 as y
  on x.id = y.id;
quit;

* view contents of inner join
  and view in data viewer;
proc contents data=dat12_inner;
run;
```

To perform an OUTER JOIN in SAS using PROC SQL:

```
* perform outer join;
proc sql;
  create table dat12_outer as
  select * from rforsas.dat1 as x full join rforsas.dat2 as y
  on x.id = y.id;
quit;

* view contents of outer join
  and view in data viewer;
proc contents data=dat12_outer;
run;
```

R code to concatenate (bind rows)

To add or merge additional rows to our dataset, we need match up the columns by their variable names. The variable names need to match for this to work. So, let's check the variables names in our two datasets `treat1` and `treat2`.

```
# check variable names
names(treat1)
```

```
[1] "id"    "treat" "age"   "cesd"  "pcs"
```

```
names(treat2)
```

```
[1] "id"    "treat" "age"   "cesd"  "mcs"
```

Since the variable names match, we can easily align the new rows from `treat2` and add them to `treat1` using `bind_rows()` from `dplyr`.

```
# stack datasets - join by variables
stack12 <- bind_rows(
  x = treat1,
  y = treat2
)
head(stack12)
```

	id	treat	age	cesd	pcs	mcs
1	3	0	26	39	74.80633	NA
2	4	0	39	15	61.93168	NA
3	5	0	32	39	37.34558	NA
4	7	0	49	52	24.51504	NA
5	9	0	50	50	38.27088	NA
6	12	0	58	49	41.93376	NA

SAS code to concatenate (bind rows)

We can simply perform dataset concatenation in SAS using the DATA statement.

```
* stack 2 datasets
   using concatenation;

data stack12;
    set rforsas.treat1 rforsas.treat2;
run;

* view contents of stacked dataset
  and view in data viewer;

proc contents data=stack12;
run;
```

3. Restructuring Datasets

One last bit of data wrangling that is helpful to know is how to “restructure” datasets for moving data in columns to row and visa-versa.

Learn more about “pivoting” data in [Chapter 5.3.2 How does pivoting work?](#) in “R for Data Science (2e)”.

A. Restructure from WIDE to LONG

The HELP dataset is currently designed so that there is only one row for each of the 453 participants. So, the data that was collected at the 5 time points are in different columns. For example, the CESD scores which measured depressive symptoms at each time point, are currently separated out into 5 columns:

- `cesd` - CESD scores at baseline
- `cesd1` - CESD scores at 1st follow-up at 6 months
- `cesd2` - CESD scores at 2nd follow-up at 12 months
- `cesd3` - CESD scores at 3rd follow-up at 18 months
- `cesd4` - CESD scores at 4th follow-up at 24 months

Hadley Wickham would say this is “[untidy](#)” data since the CESD measure is in 5 different columns!

So, let’s tidy up this data by moving these 5 columns into 1 single column. This will also add 5 rows for each participant - one for each time point. The biggest advantage to this besides “tidying” up our data, is that we will create a NEW variable explicitly for time.

First let’s pull out the 5 CESD variables, the participant IDs and a few other static/constant variables like `age`, `female` and `treat`.

```
# pull out the cesd scores at all 5 time points
# also pull out baseline variables age and female and id
# and keep treatment group assignment

cesd5times <- helpdata %>%
  select(id,
         age,
         female,
         treat,
         cesd,
         cesd1,
         cesd2,
         cesd3,
```

```
cesd4)
```

```
# cesd5times has 453 rows and 9 columns  
dim(cesd5times)
```

```
[1] 453    9
```

```
# quick look at the data, top 10 rows  
head(cesd5times,  
      n=10)
```

	id	age	female	treat	cesd	cesd1	cesd2	cesd3	cesd4
1	1	37	0	1	49	7	NA	8	5
2	2	37	0	1	30	11	NA	NA	NA
3	3	26	0	0	39	14	NA	NA	49
4	4	39	1	0	15	44	NA	NA	20
5	5	32	0	0	39	26	27	15	28
6	6	47	1	1	6	23	NA	NA	NA
7	7	49	1	0	52	NA	47	54	52
8	8	28	0	1	32	18	NA	25	NA
9	9	50	1	0	50	33	NA	35	27
10	10	39	0	1	46	37	47	NA	NA

We will use the `pivot_longer()` function from the [tidyr package](#) to go from a WIDE to LONG formatted dataset.

For `pivot_longer()` we define 3 arguments:

- `cols = -c(id, age, female, treat)`
 - these are the columns that will NOT be restructured. They are treated as constant and kept in the resulting dataset. The - minus sign before `c()` says to ignore these columns.
- `names_to = "varnames"`
 - this captures the original variable names and saves them for reference
- `values_to = "cesd"`
 - this is the name of the NEW variable which will contain ALL of the CESD scores from all 5 time points.

```
# Restructure from WIDE to LONG =====  
# To do this, we will use  
# pivot_longer() from the tidyr package
```

```
library(tidyr)
```

```
cesd5times_long <-  
  cesd5times %>%  
  pivot_longer(  
    cols = -c(id, age, female, treat),  
    names_to = "varnames",  
    values_to = "cesd"  
  )
```

```
# The result cesd5times_long is now  
# 453*5 = 2265 rows with 6 columns  
dim(cesd5times_long)
```

```
[1] 2265    6
```

```
# quick look at the data  
head(cesd5times_long,  
     n=10)
```

```
# A tibble: 10 x 6
      id   age female treat varnames  cesd
  <int> <int>   <int> <int> <chr>    <int>
1     1     37      0     1  cesd     49
2     1     37      0     1  cesd1      7
3     1     37      0     1  cesd2     NA
4     1     37      0     1  cesd3      8
5     1     37      0     1  cesd4      5
6     2     37      0     1  cesd     30
7     2     37      0     1  cesd1     11
8     2     37      0     1  cesd2     NA
9     2     37      0     1  cesd3     NA
10    2     37      0     1  cesd4     NA
```

We still do not have a variable yet for TIME, so let's make one. We will use the `dplyr::mutate()` function along with the `case_when()` function from `dplyr` to re-code the `varnames` into time index numbers. And then I took those index numbers to create a factor type variable for time as well with better labels.

```
# To add a time point, we could do the following:
# - first create a new time point index
# - add a factor type variable for time with labels
# - learn the case_when() function
cesd5times_long <-
  cesd5times_long %>%
  mutate(
    timeindex = case_when(
      varnames == "cesd" ~ 1,
      varnames == "cesd1" ~ 2,
      varnames == "cesd2" ~ 3,
      varnames == "cesd3" ~ 4,
      varnames == "cesd4" ~ 5,
      .default = NA_real_
    ),
    time.f = factor(
      timeindex,
      levels = c(1, 2, 3, 4, 5),
      labels = c("Baseline",
                 "6 mo",
                 "12 mo",
                 "18 mo",
                 "24 mo")
    )
  )
head(cesd5times_long, n=10)
```

```
# A tibble: 10 x 8
   id   age female treat varnames  cesd timeindex time.f
<int> <int> <int> <int> <chr>    <int>    <dbl> <fct>
1     1    37     0     1  cesd      49      1 Baseline
2     1    37     0     1  cesd1       7      2 6 mo
3     1    37     0     1  cesd2      NA      3 12 mo
4     1    37     0     1  cesd3       8      4 18 mo
5     1    37     0     1  cesd4       5      5 24 mo
6     2    37     0     1  cesd      30      1 Baseline
7     2    37     0     1  cesd1      11      2 6 mo
```

8	2	37	0	1	cesd2	NA	3 12 mo
9	2	37	0	1	cesd3	NA	4 18 mo
10	2	37	0	1	cesd4	NA	5 24 mo

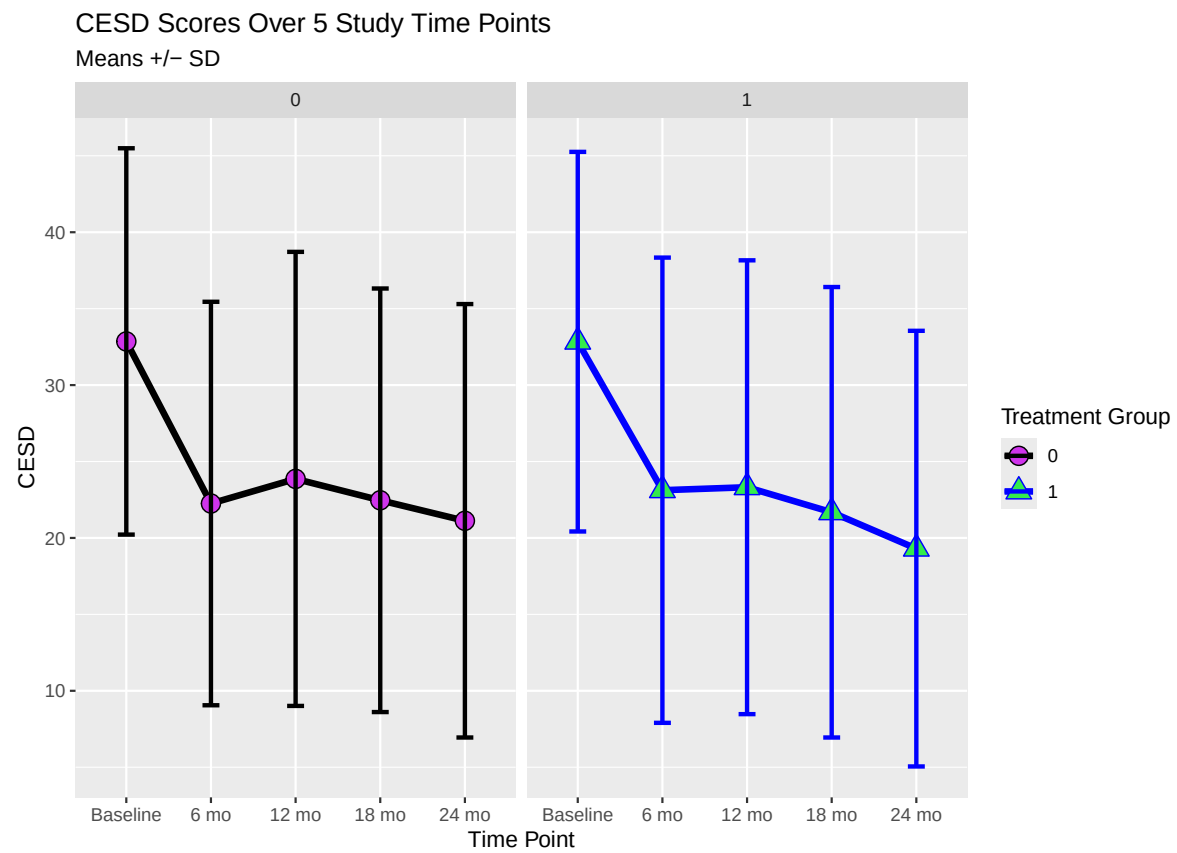
Now that we have a long formatted dataset, we can make a plot of the CESD scores over TIME by TREATMENT group. Similar to the errorbar plot example we did yesterday, we need to first get the summary statistics. And then use these to make the plot.

```
# get summary stats for CESD score
# by time and treatment group
cesd.summary <- cesd5times_long %>%
  group_by(time.f, treat) %>%
  summarise(
    sd = sd(cesd, na.rm = TRUE),
    cesd = mean(cesd, na.rm = TRUE)
  )
cesd.summary
```

```
# A tibble: 10 x 4
# Groups:   time.f [5]
   time.f   treat    sd  cesd
   <fct>   <int> <dbl> <dbl>
1 Baseline     0  12.6  32.9
2 Baseline     1  12.4  32.8
3 6 mo         0  13.2  22.3
4 6 mo         1  15.2  23.1
5 12 mo        0  14.9  23.9
6 12 mo        1  14.8  23.3
7 18 mo        0  13.9  22.5
8 18 mo        1  14.7  21.7
9 24 mo        0  14.2  21.1
10 24 mo       1  14.2  19.3
```

Make the error bar plot using the code from yesterday. I removed the `geom_jitter()` since there are so many data points for this dataset. So, I'm only showing the errorbars here.

```
# adapt the code from earlier module
library(ggplot2)
ggplot(cesd5times_long,
       aes(x=time.f,
           y=cesd,
           color = as.factor(treat))) +
  geom_line(data = cesd.summary,
            aes(group = 1),
            linewidth = 1.5) +
  geom_point(data = cesd.summary,
             size = 4,
             aes(color = as.factor(treat),
                 shape = as.factor(treat),
                 fill = as.factor(treat))) +
  geom_errorbar(aes(ymin = cesd-sd,
                    ymax = cesd+sd,
                    width = 0.2,
                    linewidth = 1,
                    data = cesd.summary) +
  scale_shape_manual(values = c(21, 24)) +
  scale_color_manual(values = c("black",
                                "blue")) +
  scale_fill_manual(values = c("#cf34eb",
                               "#34eb4c")) +
  facet_wrap(vars(treat)) +
  labs(x = "Time Point",
       y = "CESD",
       title = "CESD Scores Over 5 Study Time Points",
       subtitle = "Means +/- SD",
       caption = "",
       color = "Treatment Group",
       shape = "Treatment Group",
       fill = "Treatment Group")
```



SAS Code to Restructure from WIDE to LONG

```
* Currently the help dataset is in a WIDE format.
  Let's pull out the cesd scores at all 5 time points
  also pull out baseline variables age and female and id
  and keep treatment group assignment;

data cesd5times;
  set help;
  keep id age female treat cesd cesd1 cesd2 cesd3 cesd4;
run;

* cesd5times has 453 rows and 9 columns
# this is currently a "WIDE" data format
# where the measurements for different
# time points are in different columns
# Let's create a LONG formatted dataset
# where each ID will have a separate row
# for each time point - although for now
# we just have the original variable names

# Restructure from WIDE to LONG =====
# To do this, we will use proc transpose;

* use proc sort by id FIRST;
proc sort data=cesd5times out=cesd5times_sortid;
  by id;
run;

* use proc transpose on data sorted by id;
proc transpose data=cesd5times_sortid
  ↪ out=cesd5times_long(rename=(col1=cesdscore));
  by id age female treat;
run;

* The result cesd5times_long is now
  453*5 = 2265 rows with 6 columns;

proc contents data=cesd5times_long; run;

* add timeindex using IF THEN statements;
data cesd5times_long;
  set cesd5times_long;
```

```
if _NAME_ = "CESD" then timeindex = 1;  
if _NAME_ = "CESD1" then timeindex = 2;  
if _NAME_ = "CESD2" then timeindex = 3;  
if _NAME_ = "CESD3" then timeindex = 4;  
if _NAME_ = "CESD4" then timeindex = 5;  
run;
```

B. Restructure from LONG to WIDE

Finally, if we start with LONG formatted data and want to go back to WIDE, we can use `pivot_wider()` from the `tidyr` package.

```
# Restructure from LONG to WIDE =====
# What if we need to go back the other way
# from LONG to WIDE - we will use
# FIRST remove the timeindex and time.f
# pivot_wider() from tidyr

cesd5times_wide <-
  cesd5times_long %>%
  select(-timeindex, -time.f) %>%
  pivot_wider(
    names_from = "varnames",
    values_from = "cesd"
  )

head(cesd5times_wide, 10)
```

```
# A tibble: 10 x 9
   id   age female treat  cesd cesd1 cesd2 cesd3 cesd4
  <int> <int>   <int> <int> <int> <int> <int> <int> <int>
1     1    37     0     1    49     7    NA     8     5
2     2    37     0     1    30    11    NA    NA    NA
3     3    26     0     0    39    14    NA    NA    49
4     4    39     1     0    15    44    NA    NA    20
5     5    32     0     0    39    26    27    15    28
6     6    47     1     1     6    23    NA    NA    NA
7     7    49     1     0    52    NA    47    54    52
8     8    28     0     1    32    18    NA    25    NA
9     9    50     1     0    50    33    NA    35    27
10    10    39     0     1    46    37    47    NA    NA
```

SAS Code to Restructure from LONG to WIDE

```
* Restructure from LONG to WIDE =====
# What if we need to go back the other way
# from LONG to WIDE we can still use proc transpose;

* It would be good to resort by id and timeindex first
  to put the columns back into a logical order;

proc sort data=cesd5times_long out=cesd5times_longsort;
  by id timeindex;
run;

proc transpose data=cesd5times_longsort out=cesd5times_wide prefix=cesd_;
  by id age female treat;
  id timeindex;
  var cesdscore;
run;
```

IF TIME - %let, Duplicates, Aggregation and Macros

SAS Code

- Assign values and use
 - [letexample.sas](#)
- Find duplicate rows
 - [finddups.sas](#)
- Compute aggregate statistics
 - [aggstats.sas](#)
- Create macro functions and use
 - [macroreg.sas](#)

R Code

- Assign values and use
 - [letexample.R](#)
 - Find duplicate rows
 - [finddups.R](#)
 - Compute aggregate statistics
 - [aggstats.R](#)
 - Create macro functions and use
 - [macroreg.R](#)
-

THANK YOU AND REMINDER

I enjoyed getting to meet you during this 2-day course. Please post follow-up questions in the slack channel <https://r-for-sas-users-sh.slack.com>.

Also - **Please** fill out the course evaluation from Statistical Horizons. I and Statistical Horizons look forward to hearing your feedback!

R and SAS Code For This Module

- [module_d2_01.R](#)
- [sascode_d2_01.sas](#)

References

- Arel-Bundock, Vincent. 2022. “modelssummary: Data and Model Summaries in R.” *Journal of Statistical Software* 103 (1): 1–23. <https://doi.org/10.18637/jss.v103.i01>.
- Arel-Bundock, Vincent. 2026. *Modelssummary: Summary Tables and Plots for Statistical Models and Data: Beautiful, Customizable, and Publication-Ready*. <https://modelssummary.com>.
- Ben-Shachar, Mattan S., Daniel Lüdtke, and Dominique Makowski. 2020. “effectsize: Estimation of Effect Size Indices and Standardized Parameters.” *Journal of Open Source Software* 5 (56): 2815. <https://doi.org/10.21105/joss.02815>.
- Ben-Shachar, Mattan S., Dominique Makowski, Daniel Lüdtke, et al. 2026. *Effectsize: Indices of Effect Size*. <https://easystats.github.io/effectsize/>.
- Fox, John. 2003. “Effect Displays in R for Generalised Linear Models.” *Journal of Statistical Software* 8 (15): 1–27. <https://doi.org/10.18637/jss.v008.i15>.
- Fox, John, and Jangman Hong. 2009. “Effect Displays in R for Multinomial and Proportional-Odds Logit Models: Extensions to the effects Package.” *Journal of Statistical Software* 32 (1): 1–24. <https://doi.org/10.18637/jss.v032.i01>.
- Fox, John, and Sanford Weisberg. 2018. “Visualizing Fit and Lack of Fit in Complex Regression Models with Predictor Effect Plots and Partial Residuals.” *Journal of Statistical Software* 87 (9): 1–27. <https://doi.org/10.18637/jss.v087.i09>.
- Fox, John, and Sanford Weisberg. 2019. *An r Companion to Applied Regression*. 3rd ed. Sage. <https://socialsciences.mcmaster.ca/jfox/Books/Companion/index.html>.
- Fox, John, Sanford Weisberg, and Brad Price. 2026. *carData: Companion to Applied Regression Data Sets*. <https://r-forge.r-project.org/projects/car/>.
- Fox, John, Sanford Weisberg, Brad Price, Michael Friendly, and Jangman Hong. 2022. *Effects: Effect Displays for Linear, Generalized Linear, and Other Models*. <https://www.r-project.org>.

- Hebbali, Aravind. 2024. *Olsrr: Tools for Building OLS Regression Models*. <https://olsrr.rsquaredacademy.com/>.
- Iannone, Richard, Joe Cheng, Barret Schloerke, et al. 2026. *Gt: Easily Create Presentation-Ready Display Tables*. <https://gt.rstudio.com>.
- Lenth, Russell V. 2024. *Emmeans: Estimated Marginal Means, Aka Least-Squares Means*. <https://rvlenth.github.io/emmeans/>.
- R Core Team. 2026. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. <https://doi.org/10.32614/R.manuals>.
- Robinson, David, Alex Hayes, Simon Couch, and Emil Hvitfeldt. 2026. *Broom: Convert Statistical Objects into Tidy Tibbles*. <https://broom.tidymodels.org/>.
- Sjoberg, Daniel D., Joseph Larmarange, Michael Curry, et al. 2025. *Gtsummary: Presentation-Ready Data Summary and Analytic Result Tables*. <https://github.com/ddsjoberg/gtsummary>.
- Sjoberg, Daniel D., Karissa Whiting, Michael Curry, Jessica A. Lavery, and Joseph Larmarange. 2021. “Reproducible Summary Tables with the Gtsummary Package.” *The R Journal* 13: 570–80. <https://doi.org/10.32614/RJ-2021-053>.
- Warnes, Gregory R., Ben Bolker, Thomas Lumley, et al. 2022. *Gmodels: Various r Programming Tools for Model Fitting*. <https://doi.org/10.32614/CRAN.package.gmodels>.
- Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>.
- Wickham, Hadley, Winston Chang, Lionel Henry, et al. 2025. *Ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics*. <https://ggplot2.tidyverse.org>.
- Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, and Davis Vaughan. 2026. *Dplyr: A Grammar of Data Manipulation*. <https://dplyr.tidyverse.org>.
- Wickham, Hadley, Davis Vaughan, and Maximilian Girlich. 2025. *Tidyr: Tidy Messy Data*. <https://tidyr.tidyverse.org>.

Other Helpful Resources

[Other Helpful Resources](https://tinyurl.com/r4sasusers)